

Test de SR-TE sur trois plateformes et comparaison : CISCO Devnet Sandbox, CML-free et ContainerLab (SR-Linux)

Encadrant : **Naceur MALOUCHE,**

Étudiants : **Loïc HUANG, Qinghui LIN, Oum
LAMRABET, Ines HARRAOUI**

Table des matières

1	Cahier des charges	2
2	Plan de développement	4
3	Analyse	7
4	Conception	12
5	Critères de comparaison	16
6	Compte rendu	16
7	Bibliographie	36
8	Annexe ContainerLab	41
9	Annexe IOS XR	44

1 Cahier des charges

1.1 Introduction

Ce projet s'inscrit dans le cadre de l'évolution des architectures de réseaux de transit vers des solutions plus programmables et flexibles. Il porte sur l'étude et la mise en œuvre du Segment Routing Traffic Engineering (SR-TE), une technologie clé pour le contrôle des chemins réseau de transit.

1.2 Présentation du projet

Ce projet porte sur l'étude et la mise en œuvre du Segment Routing Traffic Engineering (SR-TE) dans plusieurs environnements d'expérimentation. Le but est de tester SR-MPLS-TE, la technologie SR-TE s'appuyant sur le mécanisme de transport MPLS (Multiprotocol Label Switching), sur trois plateformes différentes, tout en comparant leur fonctionnement, leur complexité de configuration et leurs fonctionnalités proposées.

1.2.1 Contexte et problématique

Le Segment Routing est devenu la technologie de référence pour la transmission de paquets dans les réseaux modernes. De nombreux opérateurs envisagent de migrer leurs infrastructures actuelles, notamment sous MPLS-TE, vers SR-TE pour gagner en flexibilité. MPLS-TE est le Traffic Engineering (TE) classique utilisant MPLS, TE permet de contrôler le trafic des paquets selon des contraintes établies selon nos besoins. En effet, SR-TE s'affranchit de la contrainte sur le maintien des états des tunnels de routage, créés par les routeurs par lesquels les paquets doivent passer, indispensable sous MPLS-TE. SR-TE réduit ainsi la charge de calculs sur les routeurs du réseau.

Le but est alors de pouvoir vérifier les capacités proposées par SR-TE en établissant un laboratoire de test, simulant un réseau implémentant SR-TE.

1.2.2 Objectifs du projet

Trois environnements d'expérimentation nous sont proposés, tous utilisant un type d'émulation différent : cloud distant, virtualisation locale, conteneurisation. L'objectif final est de comparer les différentes plateformes, selon différentes contraintes, et d'en choisir le meilleur permettant d'établir un laboratoire SR-TE.

Les objectifs principaux du projet consistent à tester SR-MPLS-TE sur les trois plateformes distinctes, comparer la facilité de configuration des laboratoires de tests et les fonctionnalités offertes sur ces plateformes, et enfin produire une machine virtuelle regroupant l'ensemble des tests réalisés.

1.2.3 Périmètre et contraintes

Le projet se limite à l'utilisation et à la comparaison des trois plateformes suivantes sous AlmaLinux :

- Cisco DevNet Sandbox : plateforme Cloud accessible à distance, proposant des routeurs Cisco IOS-XR ;
- Cisco Modeling Labs (CML) Free : plateforme locale proposant des routeurs Cisco IOS-XE ;
- ContainerLab : plateforme locale utilisant des NOS (Network Operating System), logiciels spécialisés pour gérer les ressources réseau, conteneurisés.

Le projet nous contraint d'effectuer les tests de plateformes sous une machine virtuelle AlmaLinux, dont le disque virtuel final doit être fourni au format .vmdk. La topologie du réseau testé sur les plateformes doit être clairement définie. De plus, le protocole de routage interne, utilisé pour partager la topologie du réseau entre les routeurs, doit être OSPF (Open Shortest Path First) et son extension OSPF-TE, implémentant l'ingénierie de trafic.

1.3 Descriptif du contenu final attendu

Émulation de réseaux : pouvoir créer et lancer un réseau virtuel sur les trois plateformes.

Ingénierie de trafic : définir des chemins explicites et dynamiques de routage de paquets via des politiques de routage basées sur des contraintes réseau (bande passante par exemple).

Tableau comparatif : avoir un tableau comparatif des différentes plateformes basé sur des critères selon la facilité de configuration des laboratoires, les ressources exigées pour utiliser les plateformes et les fonctionnalités proposées.

Laboratoire final : délivrer un laboratoire final implémentant le Segment Routing Traffic Engineering utilisant MPLS, avec toutes ses fonctionnalités, utilisant OSPF-TE comme protocole de routage interne, et implémentant une fonctionnalité de délégation du calcul de chemins à un routeur externe.

2 Plan de développement

Pour organiser le travail de manière efficace, nous avons divisé le projet en plusieurs phases :

2.1 Phase 1 : Préparation et prise en main du sujet

Cette phase permet de découvrir de nouvelles technologies que nous ne connaissons pas encore et de préparer l'environnement pour pouvoir tester les différentes plateformes. Elle est divisée en deux tâches :

Tâche 1.1 : Recherche et étude du sujet

- Analyse du fonctionnement du Segment Routing et de l'architecture des politiques de routage (SR Policies).
- Étude des extensions d'OSPF pour l'ingénierie de trafic (OSPF-TE).
- Étude du protocole PCEP (Path Computation Element Communication Protocol) utilisé pour la délégation du calcul de chemins.

Tâche 1.2 : Mise en place de l'environnement de travail

- Installation de la machine virtuelle AlmaLinux.
- Mise en place des outils essentiels (Docker, Git, etc.)

2.2 Phase 2 : Installation et Configuration

Cette phase permet de se familiariser avec les différentes plateformes afin de préparer une topologie de test.

Tâche 2.1 : Prise en main des différentes plateformes

- Installation et étude du fonctionnement de ContainerLab, Sandbox Cisco DevNet et de CML Free.

Tâche 2.2 : Configuration du protocole OSPF et validation de la topologie

- Sélection et validation de la topologie réseau.
- Configuration du protocole OSPF et activation des extensions OSPF-TE sur chaque routeur.

2.3 Phase 3 : Tests SR-TE

Tâche 3.1 : Implémentation du SR-MPLS-TE

- Configuration des identifiants des routeurs nécessaires au Segment Routing (Segment Identifiers) et création de tunnels SR-TE utilisant des politiques de routage (SR Policies).
- Test de l'utilisation du protocole PCEP.

Tâche 3.2 : Exécution des scénarios de test et débogage

Deux scénarios ont été testés :

- Forcer un chemin avec les SR Policies.
- Reroutage dynamique lorsqu'un lien tombe en panne.

Tâche 3.3 : Collecte des données

Pour chaque plateforme, les données suivantes sont collectées :

- **Consommation de ressources** : CPU, mémoire.
- **Environnement** : Vitesse et facilité de déploiement des labs, nécessité de licences, similitude au niveau des commandes, etc...

2.4 Phase 4 : Finalisation et Livraison

Tâche 4.1 : Analyse

- Analyse des données afin de déterminer les avantages et inconvénients de chaque plateforme selon les critères définis.

Tâche 4.2 : Rendu

- Nettoyage de la machine virtuelle AlmaLinux.
- Exportation et compression de la VM au format `.vmdk`
- Rédaction du rapport final de projet.

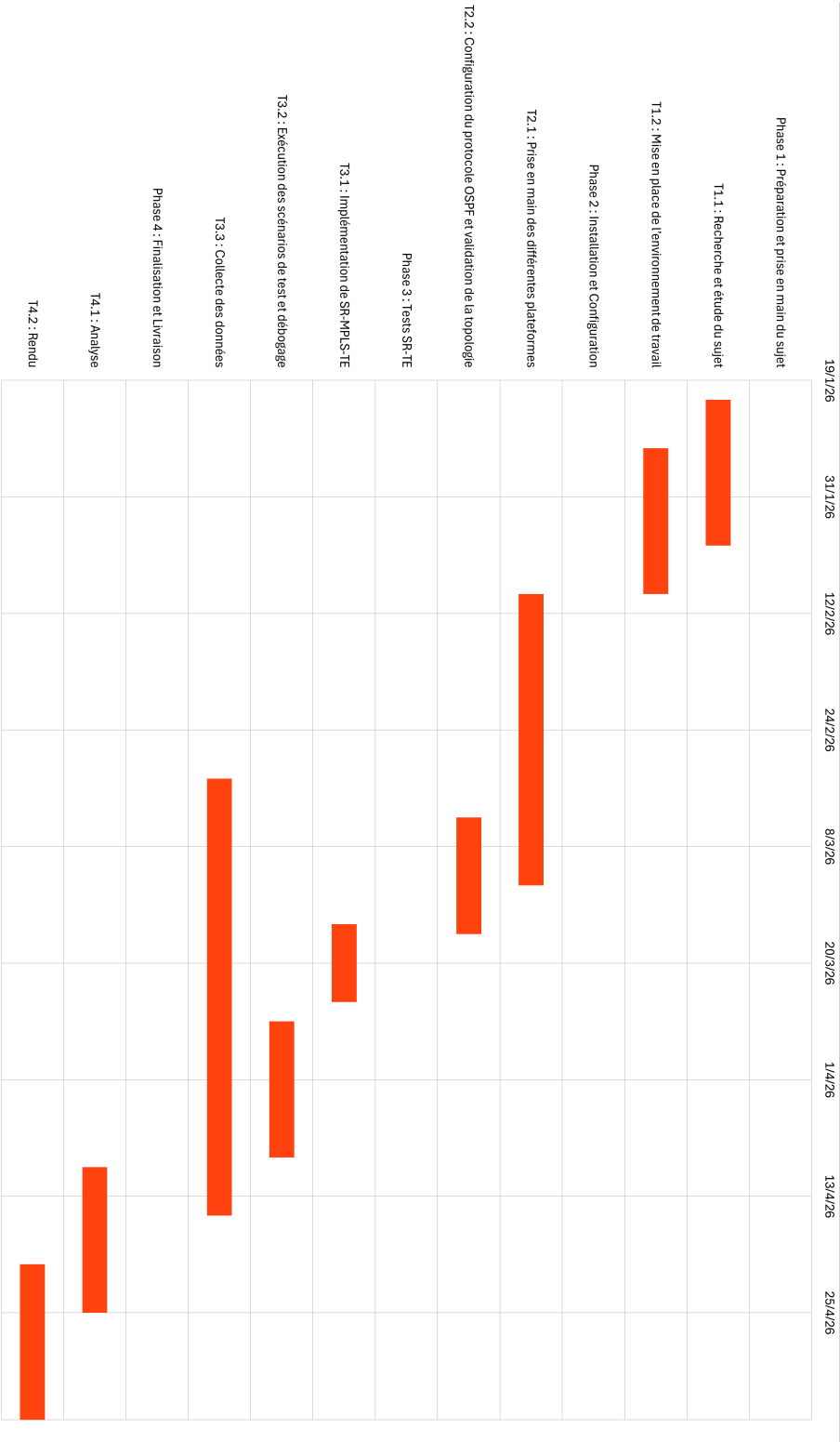


Figure 1 : Diagramme de Gantt

3 Analyse

Cette section est consacrée à l'explication des différentes notions techniques nécessaires à la compréhension du sujet, à commencer par le Segment Routing.

3.1 Open Shortest Path First

OSPF est un protocole de routage interne utilisant IP permettant d'établir la topologie du réseau et de calculer un plus court chemin entre chaque routeur du réseau via l'algorithme SPF (Shortest Path First).

Chaque routeur du réseau établit une relation d'adjacence avec ses routeurs voisins, et communique ensuite la liste des réseaux auxquels il est connecté en envoyant des messages Link-State Advertisements (LSA) contenant des informations sur le réseau, qui se propagent de proche en proche.

Avec l'ajout de nouveaux protocoles nécessitant plus d'informations, vient l'ajout des Opaque LSAs. Ce nouveau type de message est une extension des messages LSA, rajoutant d'autres types d'informations à propager dans le réseau (comme la SRGB ou les Node-SID, Adjacency-SID, vus plus loin).

3.2 Segment Routing

Le Segment Routing est une nouvelle technologie de routage où le chemin d'un paquet est défini par une liste de segments (instructions) dans l'en-tête du paquet lui-même. Cette idée reprend le concept du Source Routing, dans lequel le routeur émetteur spécifie l'itinéraire du paquet à travers le réseau.

Un segment est une instruction de routage appelée Segment Identifier (SID). Un SID est associé à deux types d'identifiants : l'identifiant d'un routeur unique ou d'une interface de sortie, respectivement nommés Node-SID et Adjacency-SID.

Par exemple, si l'on souhaite envoyer un paquet vers un routeur spécifique, le segment Node-SID permet de spécifier aux routeurs du réseau que le paquet doit être acheminé vers un routeur spécifique. Le segment Adjacency-SID permet de forcer la sortie d'un paquet du routeur via une interface spécifique. L'Adjacency-SID est une instruction locale à un routeur.

On peut également différencier ces types de segments selon deux catégories :

- **Globaux** (Node-SID) : chaque nœud du réseau annonce son identifiant, unique dans le domaine de routage, en passant par le protocole de routage interne. Tous les nœuds ont ainsi connaissance du Node-SID de chaque nœud du réseau.
- **Locaux** (Adjacency-SID) : ils n'ont de sens que pour le nœud qui les a instanciés et désignent une instruction de sortie d'interface réseau spécifique au nœud.

3.3 Traffic Engineering

Le Traffic Engineering (TE), ou ingénierie de trafic, regroupe un ensemble de mécanismes permettant d'optimiser l'utilisation des ressources réseau. Son but est d'éviter la congestion sur certains liens tout en sous-utilisant d'autres, et d'améliorer la qualité de service (QoS) en calculant des chemins qui respectent des contraintes précises (bande passante disponible, latence, etc.).

3.4 SR-TE

Le SR-TE (Segment Routing Traffic Engineering) est une évolution du Segment Routing en appliquant les principes du Traffic Engineering. Pour cela, il rajoute des préférences de routage (latence, bande passante, etc.), contrairement à des protocoles de routage qui sont uniquement basés sur le plus court chemin comme OSPF et IS-IS (Intermediate System to Intermediate System).

Dans l'architecture SR-TE, le trafic est géré par des Segment Routing Policies (SR Policies). Une SR Policy correspond à un ensemble de chemins candidats (Candidate Paths), chacun implémentant une politique (ou règle) de routage différente, auxquels on attribue des listes de segments (Segment Lists). Les chemins candidats permettent d'imposer des préférences (par exemple éviter certains liens ou prendre le chemin avec la meilleure bande passante).

À chaque politique est associée une destination (*endpoint*), un nœud d'entrée (*headend*) et une préférence (*color*).

Chaque chemin candidat dans une SR Policy définit une préférence sur les listes de segments qu'ils contiennent et est identifié par une valeur de préférence (*Preference*), ainsi que d'un SID appelé *Binding-SID*. Le BSID est un SID local au routeur. À la réception d'un paquet ayant un BSID, ce dernier est remplacé par la liste de segments active associée au chemin candidat ayant ce BSID.

Ces listes de segments peuvent être définies manuellement, calculées automatiquement ou être une combinaison des deux. Chaque chemin a un poids qui lui est assigné. Lorsqu'un candidat est choisi, le trafic sera équilibré selon le poids des différentes listes de segments (load-balanced traffic).

Le calcul des chemins peut être réalisé par les routeurs ou de manière centralisée via le protocole PCEP (cf. 3.6) par un contrôleur PCE (Path Computation Element) qui communique avec un routeur PCC (Path Computation Client) du réseau.

En cas de panne d'un lien ou de latence sur le réseau. Le trafic peut être redirigé vers un autre chemin disponible sans attendre que les protocoles aient fini de calculer de nouveaux chemins. Cela permet d'avoir une meilleure stabilité du réseau et de qualité de service.

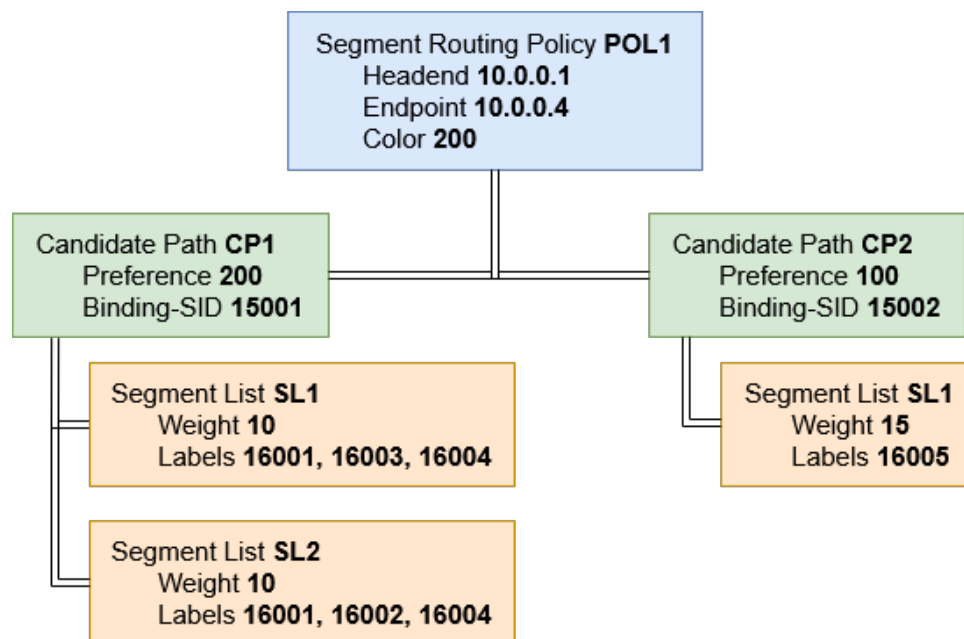


Figure 2 : Exemple de SR Policy

3.5 SR-MPLS

Le Segment Routing peut être déployé sur un plan de données MPLS (Multiprotocol Label Switching) existant, on parle alors de SR-MPLS.

MPLS est un mécanisme de transport de paquets utilisant des tunnels pour router les paquets. Les tunnels sont représentés par une pile de labels, remplaçant le routage via IP classique. Ce protocole retire la nécessité de recalculer un chemin de routeur en routeur à chaque transit du paquet. Les routeurs ont juste besoin de se référer à leur table de routage LFIB (Label Forwarding Information Base) pour connaître l'action à effectuer.

Dans SR-MPLS, la liste de segments correspond à une pile de labels MPLS, où le segment qui sera traité en premier est le label situé en haut de la pile. Les opérations sur les segments peuvent être associées aux opérations MPLS : (SR-MPLS ↔ MPLS)

- **PUSH ↔ PUSH** : empile un nouveau label en haut de la pile de labels.
- **CONTINUE ↔ SWAP** : remplace le label en haut de la pile par un autre label pour continuer à faire avancer le paquet dans le réseau.
- **NEXT ↔ POP** : retire le label en haut de pile.

Chaque nœud définit un intervalle de labels pour définir les Node-SID appelé SRGB (Segment Routing Global Block) et un intervalle pour définir les Adjacency-SID appelé SRLB (Segment Routing Local Block). Il est nécessaire d'avoir les mêmes intervalles pour tous les nœuds afin d'avoir une cohérence des segments sur le réseau. Cet intervalle permet de convertir les index des nœuds en labels MPLS.

Pour obtenir le label MPLS à utiliser afin d'atteindre un nœud, les autres nœuds du réseau doivent additionner la valeur de la borne inférieure du SRGB et l'index associé au nœud destinataire. Par exemple, si le SRGB commence à 16000 et que l'index de destination est 3, le routeur génère localement le label 16003. Comme tous les nœuds utilisent le même SRGB, ce label devient une instruction comprise par l'ensemble du réseau pour diriger le trafic.

Prenons un autre exemple en considérant que les Node-SID sont déjà calculés sur chaque routeur avec une borne inférieure de SRGB commençant à 16000. Si nous souhaitons acheminer un paquet du routeur R1 vers le routeur R7 en passant par le routeur R5, il faut construire une liste de segments. Tout d'abord, nous ajoutons le Node-SID de R5 afin d'indiquer que l'on souhaite passer par ce routeur peu importe le chemin intermédiaire. Ensuite, nous ajoutons un Adjacency-SID pour forcer le passage par le lien entre R5 et R4. Enfin, nous ajoutons le Node-SID de R7, indiquant la destination finale. Grâce à la liste de segments, le chemin suivi par le paquet respecte la contrainte de passer par R5. Une représentation de l'exemple est donnée ci-dessous.

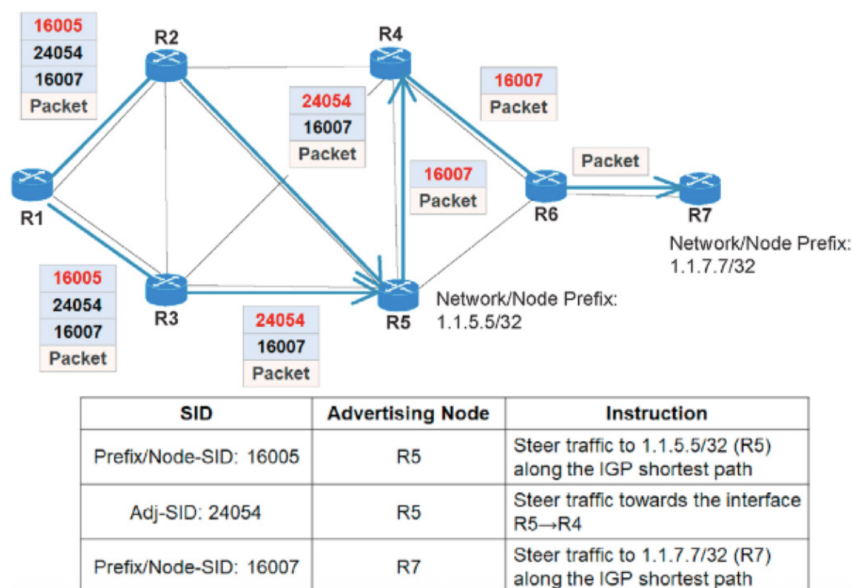


Figure 3 : Exemple de routage de paquet en utilisant les SID extrait de [8]

3.6 PCEP

Le protocole PCEP (Path Computation Element Communication Protocol) est utilisé pour la délégation de calculs de chemins. Dans ce protocole, on peut distinguer 2 rôles :

- **PCE** (Path Computation Element) : Le contrôleur qui s'occupe de calculer les chemins.

- **PCC** (Path Computation Client) : Un nœud d'entrée (ingress) du réseau qui interroge le PCE afin d'obtenir un chemin.

Le PCE apprend la topologie du réseau via BGP-LS (Border Gateway Protocol - Link State) ou directement via OSPF s'il est à l'intérieur du réseau. BGP est un protocole d'échange d'informations de routage entre systèmes autonomes (ensemble d'équipements ayant une politique de routage interne propre). Son extension BGP-LS permet d'échanger les états des liens du réseau, notamment les informations sur la bande passante. Ensuite, le PCE reçoit les informations envoyées par le PCC. Les informations peuvent être la source, la destination, la bande passante ou même une préférence. Il calcule alors un chemin et renvoie au PCC un chemin optimal sous forme de liste de segments.

Il existe deux types de fonctionnement pour le PCE :

- **Stateless** : le PCE calcule les chemins de manière indépendante et ne conserve pas les chemins qu'il a calculés.
- **Stateful** : le PCE conserve tous les chemins actifs dans une base de données. Cela permet d'avoir une vision globale du trafic et de pouvoir modifier les chemins existants si la topologie du réseau change.

3.7 Présentation des plateformes

3.7.1 ContainerLab

Avec l'augmentation de NOS (Network Operating Systems ou systèmes d'exploitation de réseau) conteneurisés, la demande pour des moyens d'établir des environnements de tests augmente. Les outils d'orchestration comme docker-compose ne permettent pas de représenter facilement des topologies réseau complètes, c'est-à-dire, un réseau de routeurs reliés par des liens.

Containerlab se présente alors comme un outil open source qui permet de créer, déployer et gérer des topologies réseau en utilisant des conteneurs Docker. Il fonctionne à partir d'un fichier de configuration au format YAML. À l'intérieur de ce fichier, on décrit une topologie avec les nœuds, les types de routeurs et les liens. Il supporte des NOS conteneurisés comme Nokia SR Linux, Arista cEOS, FRR, VyOS, RARE/freeRtr, etc.. et ne possède pas d'interface graphique.

3.7.2 CML Free

Cisco Modeling Labs Free (CML Free) est une version gratuite de la plateforme de simulation réseau Cisco Modeling Labs [20]. Le logiciel permet de créer et de lancer des topologies réseau virtuelles en local, et ce, à partir d'une interface graphique.

Dans le cadre du projet, cette plateforme représente l'environnement de virtualisation locale. Elle permet de construire une topologie avec plusieurs routeurs, de modifier les liens entre les équipements et de tester des configurations Cisco dans un environnement contrôlé par l'utilisateur.

Cependant, la version gratuite disponible possède des limitations. En effet, le nombre de nœuds disponibles est limité et le logiciel ne fournit que certaines images accessibles au grand public. Ces limites seront prises en compte lors de la comparaison avec les autres plateformes.

3.7.3 Cisco DevNet Sandbox

Cisco DevNet Sandbox est une plateforme cloud proposée par Cisco, accessible depuis la plateforme DevNet Sandbox [18]. Elle permet d'accéder à des environnements de test déjà préparés et disponibles en ligne, contenant des équipements réseau réels ou virtualisés selon le sandbox choisi.

C'est dans ce contexte que cette plateforme représente l'environnement cloud distant utilisé dans ce projet. L'outil permet d'accéder aux équipements généralement à distance, par VPN ou par SSH, sans avoir besoin d'installer toute la plateforme localement. Cela permet de tester des technologies Cisco dans un environnement proche d'un contexte opérateur ou fournisseur.

Cependant, les sandbox ont chacun leurs particularités en fonction de celui choisi. Certains proposent une topologie fixe, avec un nombre limité d'équipements, tandis que d'autres permettent davantage d'adaptation. Ces différences seront importantes dans la suite du rapport, car elles influencent directement les possibilités de test de SR-TE.

4 Conception

4.1 Environnement de travail

L'ensemble des installations et des tests sont réalisés sur une machine virtuelle sous AlmaLinux. AlmaLinux est une distribution Linux stable, fréquemment utilisée dans les environnements de test. La machine virtuelle sera livrée en fin de projet au format `.vmdk`.

4.2 Topologie

Pour tester SR-TE sur les trois plateformes (ContainerLab / CML Free / Cisco DevNet Sandbox), nous avons choisi d'appliquer 2 topologies :

- **Topologie simple** : composée d'un PCE et de 4 routeurs (PCC). Les extrémités du réseau sont représentées par deux hôtes utilisateurs, PC1 et PC2.

- **Topologie complexe** : Composée de plus de routeurs et de liens que la topologie simple. Cependant les topologies seront différentes selon les plateformes. En raison de tests différents lors de la mise en place.

L'identifiant des nœuds (associé à l'adresse Loopback) dans le réseau est défini par une adresse du type 10.0.0.x/32., où x représente l'identifiant du nœud. Pour les interfaces reliant les nœuds, nous avons utilisé un plan d'adressage simple avec le sous-réseau 192.168.XY.0/30. X et Y correspondent aux identifiants des nœuds situés aux extrémités du lien.

Le PCE est connecté à l'un des routeurs du réseau via un réseau de management. Sur ContainerLab, ce réseau utilise le plan d'adressage 172.20.20.x. Le réseau est séparé du plan de données. Cela permet d'améliorer la sécurité et de garantir une meilleure disponibilité. En cas de congestion par exemple, le PCE reste accessible.

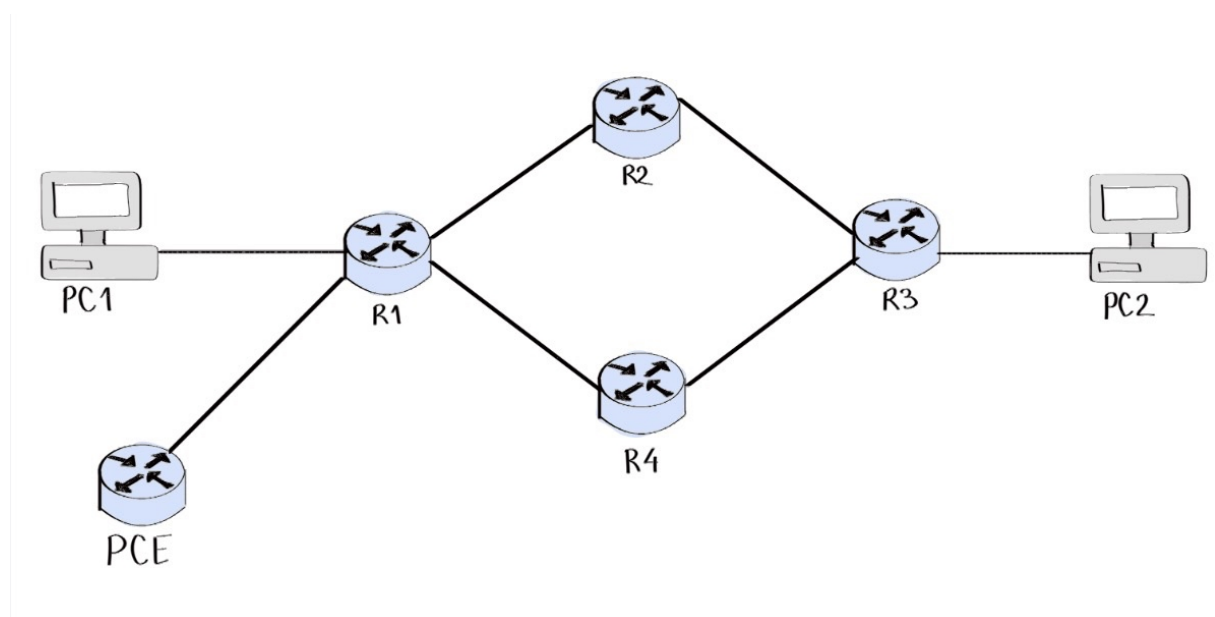


Figure 4 : Topologie simple

4.3 Choix des systèmes d'exploitation réseau

- **ContainerLab -> FRRouting (FRR)** : est un projet open-source implémentant de multiples protocoles de routage pour les plateformes Linux et UNIX. Pour les routeurs du réseau, nous avons choisi d'utiliser FRRouting. La configuration d'un routeur FRR dépend de deux fichiers :

- Le fichier `daemons` contient la configuration des démons FRR à démarrer ou non au lancement du routeur. Ces démons sont responsables de l'activation de

certaines protocoles de routage et de la communication entre démons/protocoles au sein du routeur.

- Le fichier de configuration du routeur, qui est une liste de commandes exécutées au démarrage du routeur.

Pour le plan de contrôle, deux PCE ont été envisagés :

- **RARE/freeRtr** : est un routeur open-source pouvant agir comme un PCE. Il est capable de recevoir la topologie du réseau en écoutant les échanges OSPF, et de communiquer avec les routeurs via PCEP en tant que PCE stateful. Cependant, nous n'avons pas réussi à configurer le routeur pour calculer des chemins avec la contrainte de bande passante. Il se limite à l'algorithme Shortest Path First (SPF) pour mettre en place les chemins exigés.
- **OpenDaylight (ODL)** : est un contrôleur SDN open-source supportant les protocoles BGP-LS, PCEP. Il reçoit la topologie du réseau via BGP-LS et les informations reçues sont utilisées pour construire une Traffic Engineering Database (TED), qui servira au calcul des chemins SR-TE. Cela lui permet de ne pas être directement relié au réseau. Cependant, le routeur est incompatible avec FRR car le protocole BGP-LS dans FRR est encore en phase de développement.

Ces deux solutions restent envisageables. Si d'un côté pour RARE/freeRtr la configuration du routeur pour prendre en compte des contraintes est possible, et de l'autre si l'implémentation de BGP-LS dans FRR est finalisée.

- **CML Free avec IOL XE** : Dans CML Free, plusieurs images réseau sont proposées dans l'interface, mais toutes ne sont pas utilisables avec la version gratuite [20]. Certaines nécessitent une licence ou une image de référence complète. IOL XE a donc été retenu, car c'était l'une des seules images Cisco gratuites réellement exploitables pour nos tests locaux.

Avec cette image, nous avons pu configurer les éléments de base nécessaires aux premiers tests : OSPF pour permettre aux routeurs de connaître la topologie, OSPF-TE pour ajouter des informations utiles à l'ingénierie de trafic, et SR-MPLS pour utiliser des labels associés aux routeurs.

Cependant, IOL XE reste une image limitée. Dans notre environnement, elle n'a pas permis d'utiliser correctement les SR Policies, c'est-à-dire les politiques qui servent à imposer ou à calculer un chemin SR-TE. CML Free avec IOL XE a donc été utilisé pour des tests partiels, mais n'a pas été retenu comme environnement principal pour les tests SR-TE finaux.

- **Cisco DevNet Sandbox avec IOS-XRd** : Plusieurs environnements Cisco DevNet Sandbox ont été testés pour la partie Cisco distante du projet [17]. L'objectif était de trouver un environnement permettant de tester SR-TE avec plusieurs routeurs

et plusieurs chemins possibles. Certains sandbox sont utiles pour découvrir les commandes Cisco, mais ils sont trop limités pour notre scénario, car leur topologie est souvent fixe ou composée d'un faible nombre d'équipements.

IOS-XRd a donc été retenu pour cette partie du projet. Il permet de lancer plusieurs routeurs IOS-XR sous forme de conteneurs Docker [15]. Cela nous a permis d'adapter la topologie, de créer des chemins alternatifs et d'intégrer un PCE, c'est-à-dire un contrôleur chargé de calculer des chemins.

Cet environnement a donc été utilisé comme plateforme principale pour les tests SR-TE côté Cisco. Sa principale limite concerne la contrainte de bande passante, qui n'a pas été correctement prise en compte dans la version IOS-XRd 25.3.1 utilisée. Ce point sera détaillé dans la partie réalisation.

4.4 Protocoles

Les protocoles suivants sont mis en place sur chaque plateforme :

- **OSPF** : Le protocole OSPF permet de distribuer la topologie du réseau et l'état des liens (comme la bande passante) au sein du domaine de routage.
- **SR-MPLS** : Le plan de données est défini par SR-MPLS. Chaque nœud se voit attribuer un identifiant unique (Node-SID) via l'interface Loopback. Pour chaque routeur, nous avons défini un bloc global (SRGB) de 16000 à 23999 et un bloc local (SRLB) de 15000 à 15999.
- **PCEP** : Le protocole PCEP permet la communication entre les routeurs (PCC) et le contrôleur (PCE) pour le calcul dynamique de chemins.
- **BGP-LS** : Le protocole BGP-LS (Border Gateway Protocol – Link State) permet de remonter la topologie OSPF vers le PCE afin de construire le TED pour les calculs SR-TE.

4.5 Scénarios de test

Trois scénarios principaux sont définis pour tester le fonctionnement de SR-TE sur les plateformes :

Scénario 1 — Chemin explicite : Nous configurons manuellement une SR Policy sur le routeur qui est connecté au PC1 afin de forcer le trafic à emprunter un chemin prédéfini vers PC2. La vérification est effectuée en capturant les trames circulant sur les liens à l'aide de Wireshark.

Scénario 2 — Chemin dynamique via PCE : Le routeur connecté à PC1 envoie une requête PCEP au PCE afin de calculer un chemin vers PC2. Le contrôleur PCE envoie une réponse au PCC et nous vérifions si cette réponse renvoie bien un chemin optimal en observant sa trame.

Scénario 3 — Contrainte de bande passante : Le routeur connecté à PC1 envoie une requête PCEP au PCE afin de calculer un chemin contraint par un minimum de bande passante exigé sur le chemin vers PC2. Nous vérifions à la fin que le chemin calculé respecte cette contrainte.

5 Critères de comparaison

Pour comparer les trois plateformes, nous avons défini les critères suivants :

- Facilité de mise en place des protocoles (MPLS, OSPF, PCEP, BGP-LS)
- Problèmes rencontrés lors de la configuration des plateformes
- Qualité et clarté de la documentation des plateformes
- Limites et contraintes liées à l'établissement du protocole PCEP
- Temps nécessaire à la mise en place d'un lab fonctionnel
- Complexité de la configuration d'un lab
- Similitude des commandes entre les environnements
- Contraintes sur les licences pour utiliser certains routeurs

6 Compte rendu

6.1 Déroulement du projet

6.1.1 Phase 1 : Préparation et prise en main du sujet

Nous avons commencé par clarifier ensemble les besoins du projet et définir les objectifs. Chaque membre de l'équipe a ensuite fait ses recherches sur le Segment Routing pour comprendre le sujet.

En parallèle, nous avons installé la machine virtuelle AlmaLinux, mis en place les outils de base (Docker, Git) et exploré les différents environnements de lab disponibles. Une première version du cahier des charges a été rédigée à la fin de cette phase.

Les semaines suivantes ont été consacrées à l'étude de SR-TE et d'OSPF-TE. Le cahier des charges a été mis à jour suite aux remarques de l'encadrant.

6.1.2 Phase 2 : Installation et configuration

La deuxième phase a consisté à prendre en main les trois plateformes de laboratoire : Cisco DevNet Sandbox, ContainerLab et Cisco CML. L'objectif était de vérifier, pour chacune d'elles, la possibilité de configurer un réseau, d'ajouter des serveurs et de modifier la topologie.

ContainerLab Pour ContainerLab nous avons testé plusieurs NOS :

- **Nokia SR-Linux** : Le premier NOS testé est Nokia SR-Linux. Nous nous sommes référés à la documentation de ContainerLab pour en apprendre plus sur les nœuds SR-Linux. Un fichier de configuration peut être généré automatiquement à partir de la commande :

```
clab generate --name first-test --kind nokia_srlinux \  
  --nodes 2,3 --node-prefix srl \  
  --file first-test.clab.yml \  
  --image ghcr.io/nokia/srlinux:latest
```

Commande de génération de fichier YAML

Dans ce lab, les routeurs Nokia utilisent le type `ixr-d21` par défaut. Lors des tests, nous avons très vite réalisé que certains types de routeurs SR-Linux ne supportent pas le protocole MPLS nécessaire au Segment Routing, en consultant des exemples de configuration disponibles sur le dépôt GitHub de ContainerLab [28]. Ceux qui avaient cette fonctionnalité nécessitent une licence payante. La commande `mpls` doit être accessible sous `info system` selon la documentation [24], mais elle est absente. C'est pourquoi l'utilisation de Nokia SR-Linux a été abandonnée.

- **VyOS** : Nous avons ensuite testé VyOS, un NOS open-source implémentant également le Segment Routing. VyOS propose des images qui sont mises à jour régulièrement sur leur site. Nous avons pu configurer les routeurs avec les protocoles MPLS, OSPF, et les intervalles SRGB, SRLB, pour le Segment Routing. Cependant, en consultant la documentation, nous avons constaté que VyOS ne supporte que l'algorithme Shortest Path First par défaut [41]. Le calcul de chemin sous contrainte de bande passante est alors impossible. De plus, il ne prend pas en charge le protocole PCEP. C'est pourquoi l'utilisation de VyOS a été abandonnée.
- **Arista cEOS** : Nous avons cherché un autre NOS supportant PCEP et permettant de calculer les chemins avec une contrainte de bande passante : Arista cEOS. Pour obtenir une image Arista cEOS, il est nécessaire d'avoir un compte sur leur site. Cependant, après une lecture en profondeur de la documentation, nous avons constaté que le protocole PCEP n'est pas non plus supporté [43]. C'est pourquoi l'utilisation de Arista cEOS a également été abandonnée.

- **FRRouting (FRR)** : Après plusieurs recherches, nous avons retenu FRRouting, le seul NOS testé supportant à la fois OSPF-TE, SR-MPLS et PCEP.

CML Free : CML Free a été testé comme environnement local pour la partie Cisco IOS XE du projet. La plateforme a été installée sous forme de machine virtuelle, à partir de l'image fournie pour CML Free, puis utilisée depuis son interface web. Depuis cette interface, il est possible de construire une topologie, d'ajouter des équipements, de relier les routeurs entre eux et de démarrer ou arrêter les nœuds.

L'accès aux routeurs se fait principalement depuis la console intégrée de CML. Il est aussi possible d'accéder aux consoles via le serveur console de CML, basé sur SSH. L'accès SSH direct aux routeurs dépend ensuite de la configuration réseau réalisée dans la topologie.

Après l'installation, une première topologie locale a été construite avec des routeurs IOL XE. Cette étape a permis de prendre en main l'interface CML, de vérifier le démarrage des routeurs et de préparer les configurations nécessaires aux tests suivants. Les limites rencontrées lors des tests SR-TE sont présentées dans la phase suivante.

Cisco DevNet Sandbox : Plusieurs environnements Cisco DevNet Sandbox ont été étudiés afin d'identifier une plateforme distante adaptée aux tests SR-TE. L'objectif était de disposer de plusieurs routeurs, de plusieurs chemins possibles et, si possible, d'un contrôleur PCE pour le calcul de chemins.

Les sandbox Cisco donnent accès à des environnements déjà préparés. Cependant, ils ne proposent pas tous le même niveau de liberté. Certains sont limités à un seul routeur ou à une topologie imposée, ce qui ne permet pas de reproduire un scénario SR-TE complet. Le tableau suivant résume les environnements étudiés.

Environnement testé	Topologie	Limite principale observée	Verdict
Catalyst 8000 / IOS XE	Un seul routeur réservable	Le nombre de routeurs ne permet pas de tester plusieurs chemins dans une même topologie.	Non retenu
IOS XE sur Cat8kv	Un seul routeur réservable	L'environnement permet de tester quelques commandes IOS XE, mais pas de construire un scénario SR-TE complet.	Non retenu
IOS XR Always-on	Topologie imposée	La plateforme est utile pour découvrir IOS XR, mais la topologie ne peut pas être modifiée selon les besoins du projet.	Syntaxe seulement
CML cloud dans DevNet	Topologie cloud limitée	Le support MPLS était insuffisant pour nos tests. Or, MPLS est nécessaire dans notre scénario pour transporter les paquets avec les labels SR-MPLS.	Non retenu
XRd Sandbox	Plusieurs routeurs IOS-XRd conteneurisés	La topologie peut être adaptée et permet de tester les principaux éléments attendus pour SR-TE.	Retenu

Cette comparaison a permis de justifier le choix de XRd pour la suite des tests Cisco. Les autres environnements ont été utiles pour découvrir certaines commandes ou comprendre la syntaxe Cisco, mais ils ne permettaient pas de construire librement une topologie SR-TE complète. XRd a donc été retenu car il offre plusieurs routeurs IOS-XRd, une topologie modifiable via Docker, des chemins alternatifs et la possibilité d'intégrer un PCE [16]. Les limites rencontrées avec cet environnement, notamment sur la contrainte de bande passante, sont présentées dans la partie réalisation.

6.1.3 Phase 3 : Tests SR-TE

CML Free : Les tests sur CML Free ont d'abord servi à vérifier les bases nécessaires avant de chercher à valider SR-TE complètement. Avec les routeurs IOL XE, OSPF, OSPF-TE et SR-MPLS ont pu être configurés. Les routeurs échangeaient bien leurs informations de routage, les SIDs étaient distribués et le forwarding MPLS était observable.

La limite est apparue au moment de passer aux SR Politiques. Dans notre scénario, ces politiques sont indispensables, car ce sont elles qui permettent d'imposer un chemin SR-TE ou de demander un calcul de chemin. Or, l'image IOL XE disponible dans CML Free ne permettait pas d'exploiter cette partie dans notre environnement de test.

CML Free a donc été conservé comme environnement de test partiel pour OSPF, OSPF-TE et SR-MPLS, mais il n'a pas été retenu pour la validation finale de SR-TE. Le détail des tests et la preuve CLI sont présentés dans la partie réalisation.

Problèmes rencontrés sur IOS-XRd : Sur IOS-XRd, la difficulté principale n'a pas été le lancement du sandbox, mais son adaptation au sujet. La topologie Cisco fournie au départ utilisait une organisation différente, notamment avec IS-IS, alors que le projet imposait OSPF et OSPF-TE. Les fichiers de configuration ont donc dû être repris afin de construire une topologie compatible avec notre scénario, tout en conservant les fonctions nécessaires à SR-MPLS, BGP-LS, PCEP et aux SR Politiques.

Une autre difficulté a concerné la transmission des informations de topologie vers le PCE. Certaines configurations permettaient bien d'établir les sessions entre les équipements, mais le PCE ne disposait pas toujours d'une topologie complète et exploitable pour calculer des chemins dynamiques. Cette étape de stabilisation a été nécessaire avant les tests SR-TE.

Après correction, IOS-XRd a permis de valider les fonctions principales attendues : SR-MPLS, PCEP stateful, SR Politiques explicites et SR Politiques dynamiques. En revanche, la contrainte de bande passante n'a pas pu être validée de manière fiable avec la version IOS-XRd 25.3.1 utilisée. Cette limite est analysée dans la partie réalisation.

Containerlab : FRR a été sélectionné comme routeur sur le plan de données afin d'effectuer nos tests. Les premiers tests de SR-TE ont pris du temps à débiter à cause de la prise en main des routeurs : structure utilisant des démons (processus en arrière-plan) sur FRR pour communiquer les informations entre protocoles, routeur RARE/freeRtr qui combine les fonctionnalités d'un routeur plan de données et de contrôle, OpenDaylight avec l'utilisation de l'API REST.

Nous avons pu établir des chemins statiques dans les SR Politiques, mais utilisant seulement les Node-SID. Les Segment Lists utilisant des Adjacency-SID produisent des labels en dehors de l'intervalle du SRLB. Nous n'avons pas trouvé la cause à ce problème.

Quant à l'utilisation d'un routeur sur le plan de contrôle comme PCE (Path Computation Element) pour calculer un chemin dynamiquement, le routeur RARE/freeRtr a pu calculer un chemin mais n'utilisant que l'algorithme SPF (Shortest Path First), renvoyant le chemin le plus court vers la destination. Nous n'avons pas pu avoir de confirmation sur la disponibilité de l'algorithme CSPF (Constraint Shortest Path First) pour le calcul de chemins, à cause d'une documentation incomplète.

Le routeur OpenDaylight possède toutes les fonctionnalités pour calculer un chemin selon des contraintes. Cependant, il est nécessaire d'utiliser BGP-LS afin de partager la topologie du réseau au routeur de contrôle. Or, à ce jour, FRR n'a pas encore une implémentation complète de BGP-LS permettant de partager les informations cruciales au calcul de chemins, comme les Adjacency-SID ou la SRGB et SRLB.

6.1.4 Phase 4 : Livraison

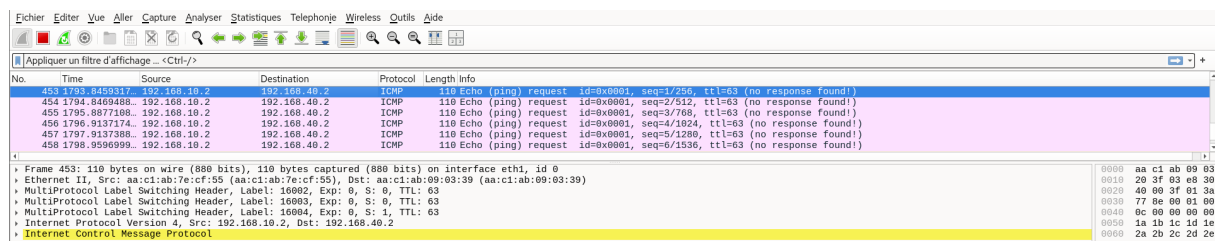
Les dernières semaines ont été consacrées à la rédaction du rapport final et au nettoyage et préparation de la version finale de la VM AlmaLinux, rendu au format `.vmdk`. La VM contient Containerlab et CML Free d'installés, ainsi que les configurations des différents labs Containerlab et Sandbox DevNet.

6.2 Réalisation

6.2.1 Containerlab : FRRouting + RARE/freeRtr

Configuration OSPF-TE Une fois que l'environnement était prêt, nous avons configuré les routeurs FRR. Chaque routeur FRR est configuré avec OSPF, avec Opaque LSA (Link-State Advertisement) pour partager les états des liens (dont la bande passante), ainsi que le Segment Routing avec la SRGB [16000, 23999] et la SRLB [15000, 15999]. MPLS-TE est également activé pour le Traffic Engineering. Après la configuration des routeurs, nous avons vérifié que des paquets OSPF ont été échangés entre les routeurs par une capture Wireshark, et que la TED a bien été établie.

Test du chemin explicite Après la configuration d'OSPF, nous avons commencé à tester le Segment Routing en ajoutant un chemin explicite au routeur d'entrée `router1`. Une SR Policy a été configurée sur `router1` afin de forcer le trafic en direction de `pc2` à emprunter le chemin `router1` → `router2` → `router3` → `router4`. La capture Wireshark confirme que les trois labels MPLS ont bien été ajoutés par-dessus l'en-tête IPv4 lors d'un ping de `pc1` vers `pc2`.



No.	Time	Source	Destination	Protocol	Length	Info
452	1763.8469317	192.168.10.2	192.168.40.2	ICMP	110	Echo (ping) request id=0x0001, seq=1/256, ttl=63 (no response found!)
453	1764.3469458	192.168.10.2	192.168.40.2	ICMP	110	Echo (ping) request id=0x0001, seq=2/256, ttl=63 (no response found!)
455	1795.8877188	192.168.10.2	192.168.40.2	ICMP	110	Echo (ping) request id=0x0001, seq=3/768, ttl=63 (no response found!)
456	1796.9137174	192.168.10.2	192.168.40.2	ICMP	110	Echo (ping) request id=0x0001, seq=4/1024, ttl=63 (no response found!)
457	1797.9137388	192.168.10.2	192.168.40.2	ICMP	110	Echo (ping) request id=0x0001, seq=5/1280, ttl=63 (no response found!)
458	1798.9596999	192.168.10.2	192.168.40.2	ICMP	110	Echo (ping) request id=0x0001, seq=6/1536, ttl=63 (no response found!)

Figure 5 : Capture Wireshark

Test des Adjacency SIDs Nous avons également testé l'utilisation des NAI (Node Address Interface) pour définir des chemins via des Adjacency SIDs. Cependant, le label 4 alloué dynamiquement se trouve en dehors de la plage SRLB [15000, 15999]. Nous n'avons pas réussi à identifier la cause du problème.

Routeur PCE : RARE/freeRtr La configuration du routeur a été faite dans le fichier `pce-sw.txt` afin de permettre au routeur d'écouter les échanges OSPF du réseau sans

y participer, ce qui lui permet de connaître la topologie. La configuration du protocole PCEP pour être un PCE se fait également dans ce fichier. Le PCE est en mode stateful. Un chemin dynamique a ensuite été demandé au PCE.

No.	Time	Source	Destination	Protocol	Length	Info
41	10.490391390	172.20.20.10	10.0.0.1	PCEP	86	Keepalive
43	10.497780131	10.0.0.1	172.20.20.10	PCEP	94	Open
44	10.498649854	172.20.20.10	10.0.0.1	PCEP	58	Keepalive
45	10.498759668	10.0.0.1	172.20.20.10	PCEP	58	Keepalive
50	10.499636419	10.0.0.1	172.20.20.10	PCEP	90	Path Computation LSP State Report (PCRpt)

Figure 6 : Établissement de la connexion PCEP

```

> Transmission Control Protocol, Src Port: 4189, Dst Port: 4189, Seq: 5, ACK: 41, Len:
  > Path Computation Element communication Protocol
    > Path Computation Reply (PCRep) Header
      > RP object
        > EXPLICIT ROUTE object (ERO)
          Object Class: EXPLICIT ROUTE OBJECT (ERO) (7)
          0001 .... = ERO Object-Type: Explicit Route (1)
          > .... 0000 = Object Header Flags: 0x0
          Object Length: 16
          > SR
            1... .... = L: Loose Hop (1)
            .010 0100 = Type: SUBOBJECT SR (36)
            Length: 12
            0001 .... = NAI Type: IPv4 Node ID (1)
            > .... 0000 0000 0001 = Flags: 0x001, SID specifies an MPLS label (M)
            > SID: 65552384 (Label: 16004, TC: 0, S: 0, TTL: 0)
            NAI (IPv4 Node ID): 10.0.0.4

```

Figure 7 : Chemin obtenu depuis le PCEP

Calcul de chemin avec contrainte de bande passante Afin de tester le calcul de chemins en fonction de la bande passante. Une limite de bande passante réservable a été définie sur les interfaces des routeurs. Ensuite, une demande de calcul de chemins a été demandée au PCE. Cependant, le PCE ne répond jamais avec un message PCRep après réception du PCRpt indiquant la bande passante exigée. Le manque de documentation de RARE/freeRtr n'a pas permis de résoudre ce problème. Nous avons donc arrêté les tests sur ce lab.

No.	Time	Source	Destination	Protocol	Length	Info
3	3.556301900	10.0.0.1	172.20.20.10	PCEP	150	Path Computation LSP State Report (PCRpt)
6	33.356096858	10.0.0.1	172.20.20.10	PCEP	58	Keepalive
9	33.357249275	172.20.20.10	10.0.0.1	PCEP	58	Keepalive
13	63.410458824	10.0.0.1	172.20.20.10	PCEP	58	Keepalive
16	63.411978953	172.20.20.10	10.0.0.1	PCEP	58	Keepalive
20	93.465527639	10.0.0.1	172.20.20.10	PCEP	58	Keepalive
23	93.466789855	172.20.20.10	10.0.0.1	PCEP	58	Keepalive
27	123.539997278	10.0.0.1	172.20.20.10	PCEP	58	Keepalive

Figure 8 : Pas de réponse au message PCRpt

6.2.2 ContainerLab : FRRouting + OpenDaylight

Configuration PCE ODL OpenDaylight utilise Karaf comme interface pour configurer le routeur. Pour récupérer la topologie du réseau, ODL doit établir une connexion BGP avec au moins l'un des routeurs du réseau. De plus, BGP-LS est nécessaire pour récupérer les informations sur les liens du réseau. Par défaut, ces fonctionnalités ne sont pas installées. Il faut installer les features **NETCONF** et **BGPCEP**. **NETCONF** est primordial pour mettre à jour la configuration des protocoles. Ce dernier utilise l'API **REST** pour communiquer avec le routeur. **BGPCEP** regroupe les fonctionnalités pour les protocoles BGP et PCEP. Les vérifications montrent l'établissement de la connexion BGP, de la connexion PCEP en mode **stateful**, ainsi que la bonne réception de la topologie OSPF et des états des liens via BGP-LS.

No.	Time	Source	Destination	Protocol	Length	Info
19	26.080200916	10.0.0.1	172.20.20.10	BGP	194	OPEN Message
31	26.100257259	10.0.0.1	172.20.20.10	BGP	85	KEEPALIVE Message
38	27.219133443	10.0.0.1	172.20.20.10	BGP	1514	UPDATE Message, UPDATE Message, UPDATE Message,
40	27.219293785	10.0.0.1	172.20.20.10	BGP	1514	UPDATE Message, UPDATE Message, UPDATE Message,
42	27.219221862	10.0.0.1	172.20.20.10	BGP	185	UPDATE Message, UPDATE Message[Malformed Packet]
54	54.009293486	10.0.0.1	172.20.20.10	PCEP	106	Open
60	54.330486858	10.0.0.1	172.20.20.10	PCEP	70	Keepalive
62	54.330595824	10.0.0.1	172.20.20.10	PCEP	102	Path Computation LSP State Report (PCRpt)
79	56.118341184	10.0.0.1	172.20.20.10	BGP	85	KEEPALIVE Message
85	84.152185784	10.0.0.1	172.20.20.10	PCEP	70	Keepalive
94	86.118423354	10.0.0.1	172.20.20.10	BGP	85	KEEPALIVE Message
102	114.128135625	10.0.0.1	172.20.20.10	PCEP	70	Keepalive
111	116.118619250	10.0.0.1	172.20.20.10	BGP	85	KEEPALIVE Message

Figure 9 : Établissement de la connexion PCEP et BGP

Calcul de chemin avec contrainte de bande passante Lors de la demande de chemin, la réponse renvoyée par le PCE indique qu'aucun chemin n'est disponible et que la source est inconnue. Dans la documentation ODL, il est dit que le routeur exige la présence du TLV SR Capabilities [45], mais cette information est absente de la topologie reçue. Un TLV contient une information Type, Length, Value, qui spécifie le type, la taille et les données échangées. Ce TLV contient la SRGB [7] nécessaire au calcul des labels, mais n'est pas encore implémenté dans FRR à ce jour (avril 2026). Nous avons pu voir que ce TLV est en cours de développement sur une branche du github de FRRouting (Pull Request #21376 [34]). Nous avons tout de même essayé d'utiliser cette version expérimentale afin de confirmer si le problème venait de ce TLV manquant. Or, ODL continue de renvoyer un chemin invalide. Les logs d'ODL indiquent des valeurs manquantes liées au Link State. Nous supposons alors que d'autres fonctionnalités clés sont manquantes.


```

▼ Path Computation Element communication Protocol
  ► Path Computation Reply (PCRep) Header
  ► RP object
  ▼ NO-PATH object
    Object Class: NO-PATH OBJECT (3)
    0001 .... = NO-PATH Object-Type: No Path (1)
    ► .... 0000 = Object Header Flags: 0x0
    Object Length: 16
    Nature of Issue: No path satisfying the set of constraints could be found (0)
    ► Flags: 0x8000
    Reserved: 0x00
  ▼ NO-PATH-VECTOR TLV
    Type: NO-PATH-VECTOR TLV (1)
    Length: 4
    .... = PCE currently unavailable: False
    .... = Unknown destination: False
    .... = Unknown source: True
    .... = BRPC Path computation chain unavailable: False
    .... = PKS expansion failure: False
    .... = No GCO migration path found: False
    .... = No GCO solution found: False
    .... = P2MP Reachability Problem: False

```

Figure 10 : Réponse du PCRep

```

opendaylight-rest@10.10.10.200 /opt/opendaylight/data/log/karaf.log | grep -i 'exception'
java.lang.NullPointerException: Cannot invoke "org.opendaylight.yang.gen.v1.urn.ietf.params.xml.ns.yang.ietf.inet.types.rev130715.AsNumber.getValue()" because the return value of "org.opendaylight.yang.gen.v1.urn.opendaylight.params.xml.ns.yang.bgp.linkstate.rev241219.linkstate.object.type.node._case.NodeDescriptors.getAsNumber()" is null
java.lang.NullPointerException: Cannot invoke "org.opendaylight.yang.gen.v1.urn.ietf.params.xml.ns.yang.ietf.inet.types.rev130715.AsNumber.getValue()" because the return value of "org.opendaylight.yang.gen.v1.urn.opendaylight.params.xml.ns.yang.bgp.linkstate.rev241219.linkstate.object.type.node._case.NodeDescriptors.getAsNumber()" is null
java.lang.NullPointerException: Cannot invoke "org.opendaylight.yang.gen.v1.urn.ietf.params.xml.ns.yang.ietf.inet.types.rev130715.AsNumber.getValue()" because the return value of "org.opendaylight.yang.gen.v1.urn.opendaylight.params.xml.ns.yang.bgp.linkstate.rev241219.linkstate.object.type.node._case.NodeDescriptors.getAsNumber()" is null
java.lang.NullPointerException: Cannot invoke "org.opendaylight.yang.gen.v1.urn.opendaylight.params.xml.ns.yang.bgp.linkstate.rev241219.linkstate.attribute.SrAttribute.getSrAdjId()" because the return value of "org.opendaylight.yang.gen.v1.urn.opendaylight.params.xml.ns.yang.bgp.linkstate.rev241219.linkstate.path.attribute.link.attributes._case.LinkAttributes.getSrAttribute()" is null
2026-04-25T18:32:09.687 | WARN | opendaylight-cluster-data-notification-dispatcher-35 | AbstractTopologyBuilder | 219 - org.opendaylight.bgpcpe.bgp-topology-provider - 1.0.0 | Transaction DOM-CHAIN-44-3 committed successfully w
hile exception captured. Rescheduling a restart of listener org.opendaylight.bgpcpe.bgp-topology.provider.LinkstateTopologyBuilder183c681f
java.lang.NullPointerException: Cannot invoke "org.opendaylight.yang.gen.v1.urn.opendaylight.params.xml.ns.yang.bgp.linkstate.rev241219.linkstate.attribute.SrAttribute.getSrAdjId()" because the return value of "org.opendaylight.yang.gen.v1.urn.opendaylight.params.xml.ns.yang.bgp.linkstate.rev241219.linkstate.path.attribute.link.attributes._case.LinkAttributes.getSrAttribute()" is null
2026-04-25T18:32:09.689 | WARN | opendaylight-cluster-data-notification-dispatcher-28 | AbstractTopologyBuilder | 219 - org.opendaylight.bgpcpe.bgp-topology-provider - 1.0.0 | Transaction DOM-CHAIN-45-0 committed successfully w
hile exception captured. Rescheduling a restart of listener org.opendaylight.bgpcpe.bgp-topology.provider.LinkstateTopologyBuilder183c681f

```

Figure 11 : Logs sur le routeur ODL

6.2.3 CML Free : tests avec IOL XE

Tests réalisés Sur CML Free, les tests ont confirmé le fonctionnement du socle OSPF, OSPF-TE et SR-MPLS avec l'image IOL XE. Les routeurs échangeaient leurs informations de routage et les labels MPLS étaient visibles dans les tables.

Limite rencontrée La limite principale est apparue au moment de tester les SR Policies. Dans notre scénario, une SR Policy est nécessaire pour imposer un chemin ou demander le calcul d'un chemin SR-TE. Or, avec l'image IOL XE 17.16.1a disponible dans CML Free, la commande attendue n'était pas reconnue :

```

R1>show version | include IOS
Cisco IOS Software [IOSXE], Linux Software (X86_64BI_LINUX-
  ADVENTERPRISEK9-M),
Version 17.16.1a, RELEASE SOFTWARE (fc1)

R1>show segment-routing traffic-eng ?
% Unrecognized command

```


Cette sortie montre que CML Free permettait de valider OSPF, OSPF-TE et SR-MPLS avec IOL XE, mais pas de créer ni d'exploiter les SR Politiques nécessaires à un scénario SR-TE complet.

Conclusion sur CML Free CML Free a donc été conservé comme environnement de test partiel et de comparaison, mais il n'a pas été retenu pour la validation finale de SR-TE.

6.2.4 Cisco DevNet Sandbox : IOS-XRd

Objectif des tests : Les tests sur IOS-XRd avaient pour objectif de vérifier le fonctionnement de SR-TE avec plusieurs routeurs, plusieurs chemins possibles et un contrôleur PCE. Cette partie présente les adaptations réalisées à partir de l'environnement fourni par Cisco, puis les validations effectuées sur les chemins explicites, les chemins dynamiques et la contrainte de bande passante.

Adaptation de la topologie Cisco : Le sandbox IOS-XRd fourni par Cisco proposait déjà une topologie de démonstration autour du Segment Routing [16]. Cette base était utile pour comprendre le fonctionnement de XRd et les outils de déploiement associés. Cependant, elle ne correspondait pas directement au scénario du projet.

En effet, la topologie initiale utilisait une organisation différente de celle attendue dans notre sujet. Elle reposait notamment sur Intermediate System to Intermediate System (IS-IS), un protocole de routage interne comparable à OSPF. Par exemple, dans un réseau composé de plusieurs routeurs, IS-IS permet à chaque routeur d'échanger des informations de topologie avec ses voisins afin de calculer le meilleur chemin vers les autres routeurs. Or, notre projet imposait l'utilisation d'OSPF et d'OSPF-TE. Les fichiers de configuration ont donc été repris afin de construire un environnement cohérent avec notre scénario de test.

La topologie adaptée pour IOS-XRd reprend le principe défini dans la partie conception : un routeur d'entrée, un routeur de sortie, deux routeurs de transit, un PCE et deux hôtes. Elle permet de disposer de deux chemins possibles entre l'entrée et la sortie du réseau, afin de vérifier si les SR Politiques orientent bien le trafic selon le chemin attendu.

Déploiement de la topologie : Le déploiement de la topologie IOS-XRd a été automatisé avec un script `deploy.sh`. Ce script prépare l'environnement, nettoie les anciens conteneurs Docker, génère le fichier `docker-compose.yml`, puis lance les routeurs IOS-XRd avec les fichiers de configuration du projet.

Cette automatisation a permis d'éviter les conflits rencontrés lors des premiers essais, notamment lorsque d'anciens conteneurs ou réseaux Docker restaient présents après l'arrêt d'une topologie. Une fois les conteneurs démarrés, les routeurs sont accessibles depuis la machine du sandbox, et les hôtes `pc1` et `pc2` permettent de tester la connectivité et le chemin suivi par le trafic.

Plan de contrôle configuré : Après le déploiement de la topologie, nous avons configuré les protocoles nécessaires aux tests SR-TE sur IOS-XRd. OSPF et OSPF-TE ont permis de diffuser la topologie du réseau, tandis que SR-MPLS a été utilisé pour associer des labels aux routeurs.

Le PCE a ensuite été intégré afin de permettre le calcul de chemins dynamiques. Dans cette organisation, `router1` porte les SR Politiques car il est le routeur d'entrée du trafic : c'est lui qui choisit le chemin à appliquer pour rejoindre `pc2`. `router4` se trouve à la sortie du réseau testé. `router2` et `router3` servent seulement de routeurs de transit : ils font passer les paquets selon les labels MPLS reçus.

SR Politiques configurées Après la mise en place du plan de contrôle, nous avons configuré plusieurs SR Politiques sur `router1`, selon la logique de configuration SR-TE décrite par Cisco [11]. Dans notre scénario, une SR Policy indique au routeur d'entrée quel chemin utiliser pour rejoindre le routeur de sortie.

Les politiques ont été distinguées par des couleurs :

- **color 100** : chemin dynamique calculé par le PCE avec la métrique OSPF classique.
- **color 128** : chemin dynamique calculé par le PCE avec un critère de latence.
- **color 200** : chemin explicite passant par `router2`.
- **color 300** : chemin explicite passant par `router3`.

Les couleurs 100 et 128 permettent donc de tester le calcul automatique par le PCE, tandis que les couleurs 200 et 300 permettent de tester un chemin imposé manuellement. Les détails de configuration et les commandes de vérification sont présentés dans l'annexe IOS XR.

Validation des politiques explicites :

Les premiers tests ont porté sur les politiques explicites. L'objectif était de vérifier si `router1` pouvait imposer un chemin précis au trafic, sans demander de calcul au PCE.

Deux chemins ont été testés :

- couleur 200 : chemin haut, de `router1` vers `router4` en passant par `router2`.
- couleur 300 : chemin bas, de `router1` vers `router4` en passant par `router3`.

Pour déclencher ces tests, la couleur annoncée par `router4` a été modifiée temporairement.

La vérification des politiques montre que la couleur 200 sort bien vers `router2`, tandis que la couleur 300 sort vers `router3` :

```
# Color 200
Outgoing Interfaces: GigabitEthernet0/0/0/0
Next Hop: 100.101.102.102 --> router2

# Color 300
Outgoing Interfaces: GigabitEthernet0/0/0/1
Next Hop: 100.101.103.103 --> router3
```

Pour vérifier le comportement réel du trafic, un traceroute a ensuite été lancé depuis pc1 vers pc2.

```
# Color 200 : chemin haut
1  router1
2  100.101.102.102 --> router2
3  100.102.104.104 --> router4
4  10.3.1.2         --> pc2

# Color 300 : chemin bas
1  router1
2  100.101.103.103 --> router3
3  100.103.104.104 --> router4
4  10.3.1.2         --> pc2
```

Ces résultats confirment que les politiques explicites fonctionnent correctement sans intervention du PCE. Le chemin est imposé par la configuration et il est bien respecté par le plan de données MPLS.

Validation des politiques dynamiques via PCE Les tests suivants ont porté sur les politiques dynamiques. L'objectif était de vérifier qu'une route annoncée avec une couleur pouvait déclencher automatiquement la création d'une SR Policy sur router1, puis demander au PCE de calculer le chemin.

Policy dynamique color 100 : Pour la couleur 100, router4 annonce la route vers le réseau de pc2 avec la communauté Color:100. Cette annonce déclenche la création automatique d'une SR Policy dynamique sur router1. La route BGP reçue montre bien l'association avec la couleur 100 :

```
Extended community: Color:100 RT:100:100
SR policy color 100, up, registered, bsid 24008
```

La policy associée est ensuite vérifiée sur router1 :

```
Color: 100, End-point: 100.100.100.104
Status:
  Admin: up  Operational: up
```

```
Preference: 100 (BGP ODN) (active)
Dynamic (pce 100.100.100.107) (valid)
Metric Type: IGP, Path Accumulated Metric: 21
SID[0]: 16104 [Prefix-SID, 100.100.100.104]
```

Cette sortie montre que la policy est active, qu'elle a été créée automatiquement par BGP ODN, et qu'elle est calculée dynamiquement par le PCE avec la métrique IGP.

La présence du chemin côté PCE a également été vérifiée avec la commande `show pce lsp`. Le LSP associé à `router1` apparaît comme actif :

```
PCC 100.100.100.101:
Tunnel Name: bgp_c_100_ep_100.100.100.104_discr_100
State: Admin up, Operation active
Setup type: Segment Routing
Binding SID: 24008
```

Enfin, un `traceroute` lancé depuis `pc1` vers `pc2` confirme le chemin réellement emprunté par le trafic :

```
1  router1.config_pc1-router1 (10.1.1.3)
2  100.101.102.102 --> router2
3  100.102.104.104 --> router4
4  10.3.1.2 --> pc2
```

Le test `color 100` valide donc le fonctionnement d'une SR Policy dynamique calculée par le PCE avec la métrique IGP. Le trafic suit le chemin passant par `router2`.

Policy dynamique color 128 La même logique a ensuite été testée avec la couleur 128. Cette fois, la route annoncée par `router4` déclenche une policy dynamique calculée avec une métrique de latence.

La route BGP reçue sur `router1` montre bien la couleur 128 :

```
Extended community: Color:128 RT:100:100
SR policy color 128, up, registered, bsid 24010
```

La policy SR-TE associée est active et calculée par le PCE :

```
Color: 128, End-point: 100.100.100.104
Status:
  Admin: up Operational: up
Preference: 100 (BGP ODN) (active)
Dynamic (pce 100.100.100.107) (valid)
Metric Type: LATENCY, Path Accumulated Metric: 20
SID[0]: 16104 [Prefix-SID, 100.100.100.104]
```

Côté PCE, la policy `color 128` est également visible dans la base des LSP. La sortie confirme que le calcul utilise bien la métrique de latence :

```
Tunnel Name: bgp_c_128_ep_100.100.100.104_discr_100
Color: 128
State: Admin up, Operation active
Setup type: Segment Routing
Binding SID: 24010
Reported path:
  Metric type: LATENCY (12), Accumulated Metric 20
Computed path: (Local PCE)
  Metric type: LATENCY (12), Accumulated Metric 20
```

Le trafic a ensuite été vérifié depuis pc1 avec un traceroute. Dans le test observé, le trafic passe par router3 :

```
1  router1.config_pc1-router1 (10.1.1.3)
2  100.101.103.103 --> router3
3  100.103.104.104 --> router4
4  10.3.1.2 --> pc2
```

Ce test montre que la policy dynamique color 128 est bien active, calculée par le PCE avec une métrique de latence, et que le trafic peut être orienté vers un chemin différent.

Conclusion : Ces tests confirment que le PCE calcule bien les politiques dynamiques avec des critères différents selon la couleur utilisée.

Vérification du fonctionnement stateful du PCE : Après la validation des politiques dynamiques, nous avons vérifié que le PCE fonctionnait bien en mode stateful. Dans ce mode, le PCE ne se limite pas à calculer un chemin : il garde aussi en mémoire l'état des chemins actifs et peut échanger des mises à jour avec les routeurs.

La session PCEP a d'abord été vérifiée côté router1 :

```
show segment-routing traffic-eng pcc ipv4 peer detail
```

La sortie montre que la session avec le PCE est active et que les capacités attendues sont présentes :

```
Peer address: 100.100.100.107
State up
Capabilities: Stateful, Update, Segment-Routing, Instantiation
Report messages: tx 5
Update messages: rx 1
```

La même vérification a été faite côté PCE. router1 et router4 apparaissent en état Up, ce qui montre que la communication avec le contrôleur est active. Les capacités affichées indiquent que le PCE peut suivre l'état des chemins et les mettre à jour. La commande `show pce lsp detail` confirme ensuite que les chemins SR-TE actifs sont bien connus par le contrôleur.

Enfin, une capture Wireshark a été réalisée pendant une coupure temporaire du lien entre router1 et router2. La capture montre des échanges de contrôle, notamment des messages PCRpt puis PCUpd, après la modification de topologie.

No.	Time	Source	Destination	Protocol	Length	Info
118	164.591295	100.100.100.101	100.100.100.107	UDP	500	UPDATE Message, UPDATE Message
119	164.592408	100.100.100.107	100.100.100.101	TCP	58	179 → 62428 [ACK] Seq=58 Ack=1069 Win=32768 Len=0
120	164.594374	100.100.100.104	100.100.100.107	BGP	500	UPDATE Message, UPDATE Message
121	164.595396	100.100.100.107	100.100.100.104	TCP	58	35064 → 179 [ACK] Seq=58 Ack=1050 Win=32768 Len=0
122	164.597814	100.100.100.107	100.100.100.104	PCEP	138	Path Computation LSP Update Request (PCUpd)
123	164.599624	100.100.100.104	100.100.100.107	PCEP	430	Path Computation LSP State Report (PCRpt)
124	164.607596	100.102.107.102	224.0.0.5	OSPF	134	LS Update
125	164.715451	100.102.107.102	224.0.0.5	OSPF	266	LS Update
126	164.734531	100.100.100.104	100.100.100.107	BGP	281	UPDATE Message
127	164.741410	100.100.100.101	100.100.100.107	BGP	281	UPDATE Message
128	164.770890	100.102.107.102	224.0.0.5	OSPF	114	Hello Packet
129	164.800481	100.100.100.107	100.100.100.104	TCP	58	4189 → 63347 [ACK] Seq=105 Ack=1513 Win=63721 Len=0
130	164.815525	100.102.107.102	224.0.0.5	OSPF	130	LS Update
131	164.935652	100.100.100.107	100.100.100.104	TCP	58	35064 → 179 [ACK] Seq=58 Ack=1277 Win=32541 Len=0
132	164.937534	100.100.100.104	100.100.100.107	BGP	185	UPDATE Message
133	164.942214	100.100.100.107	100.100.100.101	TCP	58	179 → 62428 [ACK] Seq=58 Ack=1296 Win=32541 Len=0
134	164.945558	100.100.100.101	100.100.100.107	BGP	185	UPDATE Message
135	165.138723	100.100.100.107	100.100.100.104	TCP	58	35064 → 179 [ACK] Seq=58 Ack=1408 Win=32410 Len=0
136	165.146415	100.100.100.107	100.100.100.101	TCP	58	179 → 62428 [ACK] Seq=58 Ack=1427 Win=32410 Len=0
137	165.370394	100.102.107.102	224.0.0.5	OSPF	114	Hello Packet
138	165.934973	100.102.107.102	224.0.0.5	OSPF	130	LS Update
139	165.950707	100.100.100.101	100.100.100.107	BGP	377	UPDATE Message
140	165.952845	100.100.100.104	100.100.100.107	BGP	281	UPDATE Message
141	165.968121	100.102.107.102	224.0.0.5	OSPF	266	LS Update
142	166.001159	100.102.107.102	224.0.0.5	OSPF	254	LS Update
143	166.116802	100.102.107.102	224.0.0.5	OSPF	266	LS Update
144	166.151601	100.100.100.107	100.100.100.101	TCP	58	179 → 62428 [ACK] Seq=58 Ack=1750 Win=32087 Len=0
145	166.153937	100.100.100.107	100.100.100.104	TCP	58	35064 → 179 [ACK] Seq=58 Ack=1635 Win=32183 Len=0

Figure 12 : Échanges de contrôle observés pendant un changement de topologie

Cette vérification confirme que le PCE fonctionne bien en mode stateful : il connaît les chemins actifs, reçoit les changements d'état et peut participer à la mise à jour des SR Politiques dynamiques.

Contrainte de bande passante : L'objectif de ce test était de vérifier si le PCE pouvait calculer un chemin en tenant compte d'une contrainte de bande passante minimale. Autrement dit, nous voulions voir si le contrôleur pouvait éviter un chemin lorsque les liens disponibles ne respectaient pas la bande passante demandée.

Dans un premier temps, les informations de bande passante des liens étaient bien diffusées par OSPF-TE et visibles côté PCE :

Bandwidth: Total 125000000 Bps, Reservable 12500000 Bps

Cette sortie montre que le PCE recevait bien les informations de capacité des liens. La difficulté ne venait donc pas de la remontée des informations de bande passante, mais de l'application effective d'une contrainte de bande passante sur une SR Policy dynamique.

Deux approches ont alors été testées. La première consistait à utiliser des affinités pour exclure certains liens. Cette méthode permet d'orienter le trafic vers un autre chemin, mais elle ne correspond pas à un vrai contrôle de bande passante, car elle ne réserve pas de capacité et ne vérifie pas la bande passante réellement disponible.

La seconde approche s'appuie sur la commande `override-rules` côté PCE, décrite dans la documentation Cisco dédiée aux commandes Segment Routing sur IOS XR [14]. Cette commande permet de définir des règles d'override pour des politiques initiées par un PCC, en utilisant des critères comme le peer, le nom du LSP ou la couleur de la

policy. Elle permet également d'ajouter une contrainte de bande passante avec `override constraints bandwidth`. L'objectif était donc de vérifier si cette méthode permettait au PCE de recalculer une SR Policy dynamique en tenant compte d'une bande passante minimale.

En pratique, les traces PCEP montrent cependant que la valeur de bande passante reste à 0 kbps :

```
Sending PCUpd SRTE, [PLSP-ID:3]
req-bw/res-bw:0/0 kbps

Received PCRpt.
req-bw/res-bw:0/0 kbps flags:(C:0)
```

Cette sortie montre que la contrainte de bande passante n'est pas réellement appliquée dans notre environnement de test. Le PCE et le routeur échangent bien des messages PCEP, mais aucune valeur de bande passante n'est transmise à la policy. Aucun recalcul de chemin basé sur cette contrainte n'a donc pu être validé.

Pour vérifier que le problème ne venait pas uniquement de notre configuration, la topologie de référence du sandbox a été rechargée dans un environnement propre. Le résultat obtenu était identique. Cette limite semble donc liée au comportement de la version IOS-XRd 25.3.1 utilisée dans le sandbox.

Conclusion sur IOS-XRd IOS-XRd a permis de valider une grande partie de SR-TE : SR-MPLS, OSPF-TE, BGP-LS, PCEP stateful, ainsi que les politiques explicites et dynamiques. Les tests montrent que le trafic peut être orienté manuellement ou calculé dynamiquement par le PCE.

La limite principale reste la contrainte de bande passante : les informations sont visibles côté PCE, mais la contrainte n'a pas été réellement appliquée avec `override-rules bandwidth`. Cette version permet donc de démontrer SR-TE, sans valider un contrôle strict de bande passante.

6.3 Validation

La validation des tests a reposé sur plusieurs critères techniques et pratiques. Le tableau suivant synthétise les résultats obtenus sur les trois plateformes.

Critère	ContainerLab	CML Free	Cisco DevNet Sandbox / IOS-XRd
Facilité de mise en place des protocoles	OSPF, SR-MPLS, PCEP et BGP-LS ont pu être configurés avec FRR, mais la mise en place a demandé plusieurs essais.	OSPF, OSPF-TE et SR-MPLS ont pu être testés avec IOL XE.	XRd a permis de valider des fonctions SR-TE avancées, mais seulement après adaptation de l'environnement fourni par Cisco.
Problèmes rencontrés	Difficultés liées à la configuration du PCE, aux labels SR-MPLS et aux changements d'images ou de NOS pendant le projet.	Les limites viennent surtout de l'image IOL XE disponible dans la version gratuite.	Les principales difficultés ont concerné l'adaptation de la topologie initiale, la reprise des fichiers de configuration et certaines interruptions de session pendant les tests.
Documentation	La documentation de ContainerLab et FRR est claire, mais celle de RARE/freeRtr est très limitée.	La documentation de CML est accessible, mais les possibilités réelles dépendent des images disponibles gratuitement.	La documentation Cisco est complète, mais elle demande un travail d'adaptation important pour être appliquée au contexte du sandbox gratuit.
Limites PCE / PCC	Le PCE a pu être mis en place avec OpenDaylight et RARE/freeRtr, mais les tests ont été bloqués par certaines limites de compatibilité.	Le scénario complet avec PCE et SR Policies n'a pas pu être validé.	Le PCE stateful a été validé, mais la contrainte de bande passante n'a pas pu être appliquée de manière fiable avec la version IOS-XRd utilisée.
Temps de mise en place	Plusieurs semaines ont été nécessaires à cause des problèmes d'images, de fonctionnalités indisponibles et de configuration.	Prise en main rapide grâce à l'interface graphique.	La réservation du sandbox est rapide, mais l'adaptation de la topologie et la durée limitée des sessions rendent les tests moins confortables sur la durée.
Complexité de configuration	Configuration flexible avec une topologie YAML, des fichiers par routeur et les démons FRR, mais dépendante du NOS choisi.	Configuration plus simple avec l'interface CML et les commandes IOS, mais très limitée pour SR-TE.	Configuration plus complexe, car elle repose sur IOS XR, des fichiers de démarrage et une topologie conteneurisée à adapter.

Critère	ContainerLab	CML Free	Cisco DevNet Sandbox / IOS-XRd
Commandes	Nécessite de gérer à la fois les commandes Linux, Docker, ContainerLab et les commandes des NOS. FRR reste proche de Cisco, mais avec des différences de syntaxe.	Utilise le CLI IOS classique, plus familier, avec une interface graphique CML pour gérer la topologie.	Les commandes IOS XR sont cohérentes, mais l'environnement reste moins accessible qu'une plateforme avec interface graphique dédiée.
Contraintes de licence et de session	ContainerLab, FRR et OpenDaylight sont gratuits et open source. Certains NOS comme Nokia SR-Linux peuvent cependant nécessiter une licence.	Gratuit, mais limité en nombre de routeurs et en fonctionnalités disponibles.	Gratuit via DevNet, mais limité par la durée des réservations et par la nécessité de sauvegarder régulièrement les configurations pour éviter toute perte lors d'une interruption.
Consommation de ressources	Dépend des types de NOS utilisés dans les labs, les conteneurs FRR ne consomment qu'une centaine de Mo en mémoire alors que les conteneurs Arista consomment plusieurs Go.	CML Free nécessite au moins 3 à 4 Go de mémoire pour le faire tourner.	Aucune consommation de ressources sur notre machine car la plateforme est à distance dans le cloud.
Bilan	Plateforme la plus flexible, locale et persistante. Elle permet de sortir de l'écosystème Cisco, même si tous les tests SR-TE n'ont pas pu être validés.	Plateforme utile pour des tests partiels, mais insuffisante pour valider SR-TE complètement.	Plateforme très complète pour valider SR-TE, mais moins adaptée comme environnement durable de travail à cause de sa dépendance au sandbox, de la durée limitée des sessions et de l'absence d'interface graphique dédiée.

Conclusion Chaque plateforme présente des avantages et des limites. Cisco DevNet Sandbox avec IOS-XRd offre un environnement récent et complet pour tester SR-TE, sans utiliser les ressources de notre machine locale. Cependant, les sessions sont limitées dans le temps et l'environnement doit être relancé à partir des fichiers de configuration après fermeture.

CML Free reste simple à prendre en main, mais la version gratuite n'a pas permis de

valider SR-TE complètement avec l'image IOL XE disponible.

Selon nous, la meilleure plateforme de test est ContainerLab pour sa simplicité d'utilisation et sa flexibilité. Ce dernier permet également d'utiliser des routeurs open-source sans être bloqué dans l'écosystème Cisco, malgré nos difficultés d'établissement des réseaux SR-TE.

6.4 Livraison

La VM AlmaLinux contient les installations de Docker, Containerlab, Edgeshark, Wireshark et CML Free.

Containerlab Les configurations des labs se trouvent dans le répertoire `~/Documents/Clab`. Chaque sous-répertoire contient un lab désigné par le nom du répertoire. Les labs peuvent contenir :

- Un répertoire `config` contenant la configuration de chacun des routeurs du lab (sauf pour le lab SR Linux).
- Le fichier de configuration de la topologie `xxx.clab.yml`.
- Certains labs possèdent un répertoire `topology` contenant une image de la topologie du lab.
- Un README est disponible pour les labs `freeRtr` et `OpenDaylight` contenant quelques commandes pour le lab, l'installation de Edgeshark et des documentations des routeurs.

La page d'accès à Edgeshark et aux données du routeur OpenDaylight sont mises en favoris dans le navigateur Firefox.

Sandbox DevNet / IOS-XRd : Les fichiers liés à la partie Cisco DevNet Sandbox se trouvent dans le répertoire `~/Documents/Cisco`. Ils permettent de reconstruire la topologie IOS-XRd adaptée au projet.

Ce répertoire contient :

- le fichier `docker-compose_srte.yml`, qui décrit la topologie Docker adaptée.
- le fichier `docker-compose.yml`, utilisé pour lancer la topologie.
- le script `deploy.sh`, utilisé pour préparer l'environnement et lancer la topologie.
- le fichier `README_SRTE.md`, qui résume les commandes utiles pour relancer le lab.
- les fichiers de configuration des routeurs : `router1-startup.cfg`, `router2-startup.cfg`, `router3-startup.cfg`, `router4-startup.cfg`.

- le fichier `pce-startup.cfg`, contenant la configuration du PCE.

Après connexion au VPN Cisco DevNet et à la machine virtuelle du sandbox, ces fichiers peuvent être copiés sur la machine distante. Le script `deploy.sh` permet ensuite de nettoyer les anciens conteneurs, de générer le fichier `docker-compose.yml` et de démarrer la topologie IOS-XRd.

Les fichiers livrés permettent de retrouver les adaptations réalisées pour le projet : routage, SR-MPLS, PCE, politiques explicites et politiques dynamiques.

7 Bibliographie

Références

1. Protocoles réseau et SR-TE

- [1] Ahmed Bashandy, Clarence Filsfils, Stefano Previdi, Bruno Decraene, Stephane Litkowski, and Rob Shakir. 2019. *Segment Routing with the MPLS Data Plane*. Engineering Task Force. Consulté le 27 janvier 2026. Disponible sur : <https://datatracker.ietf.org/doc/rfc8660/>
- [2] Clarence Filsfils, Darren Dukes, Stefano Previdi, John Leddy, Satoru Matsushima, and Daniel Voyer. 2020. *IPv6 Segment Routing Header (SRH)*. Engineering Task Force. Consulté le 28 janvier 2026. Disponible sur <https://datatracker.ietf.org/doc/rfc8754/>
- [3] Clarence Filsfils, Ketan Talaulikar, Daniel Voyer, Alex Bogdanov, and Paul Mattes. 2022. *Segment Routing Policy Architecture*. Engineering Task Force. Consulté le 27 janvier 2026. Disponible sur <https://datatracker.ietf.org/doc/rfc9256/>
- [4] J. P. Vasseur and Jean-Louis Le Roux. 2009. *Path Computation Element (PCE) Communication Protocol (PCEP)*. Internet Engineering Task Force. Consulté le 15 février. Disponible sur <https://datatracker.ietf.org/doc/rfc5440/>
- [5] Edward Crabbe, Ina Minei, Jan Medved, and Robert Varga. 2017. *Path Computation Element Communication Protocol (PCEP) Extensions for Stateful PCE*. Internet Engineering Task Force. Consulté le 15 février 2026. Disponible sur <https://datatracker.ietf.org/doc/rfc8231/>
- [6] Mike Koldychev, Siva Sivabalan, Samuel Sidor, Colby Barth, Shuping Peng, and Hooman Bidgoli. 2025. *Path Computation Element Communication Protocol (PCEP) Extensions for Segment Routing (SR) Policy Candidate Paths*. Internet Engineering Task Force. Consulté le 15 février 2026. Disponible sur <https://datatracker.ietf.org/doc/rfc9862/>
- [7] Stefano Previdi, Ketan Talaulikar, Clarence Filsfils, Hannes Gredler, and Mach Chen. 2021. *Border Gateway Protocol - Link State (BGP-LS) Extensions for Segment Routing*. Internet Engineering Task Force. Consulté le 18 février 2026. Disponible sur <https://datatracker.ietf.org/doc/rfc9085/>
- [8] Duo Wu and Lin Cui. 2023. A comprehensive survey on Segment Routing Traffic Engineering. *Digital Communications and Networks* 9, 4 (August 2023), 990–1008. Consulté le 28 janvier 2026. Disponible sur <https://doi.org/10.1016/j.dcan.2022.02.006>

- [9] Rob Coltun, Alex D. Zinin, Igor Bryskin, and Lou Berger. *The OSPF Opaque LSA Option*. Internet Engineering Task Force, 2008. Consulté le 31 janvier 2026. Disponible sur <https://datatracker.ietf.org/doc/rfc5250/>

Documentation Cisco

- [10] Cisco. 2024. *Segment Routing Configuration Guide, Cisco IOS XE 17 | Access and Edge Routers - Introduction to Segment Routing for the MPLS Dataplane*. Consulté le 20 février 2026. Disponible sur https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/seg_routing/configuration/xr-17/segrrt-xr-17-book/m_intro-seg-routing.html
- [11] Cisco. 2025. *Segment Routing Configuration Guide for Cisco ASR 9000 Series Routers, IOS XR Release 25.1.x, 25.2.x, 25.3.x, 25.4.x - Configure SR-TE Policies*. Consulté le 1 mars 2026. Disponible sur <https://www.cisco.com/c/en/us/td/docs/routers/asr9000/software/25xx/segment-routing/configuration/guide/b-segment-routing-cg-asr9000-25xx/configure-sr-te-policies.html>
- [12] Cisco. 2018. *Configuration de l'ingénierie de trafic de routage inter-zone (SR-TE) basée sur des éléments de calcul de chemin non-chemin (PCE)*. Consulté le 2 mars 2026. Disponible sur https://www.cisco.com/c/fr_ca/support/docs/multiprotocol-label-switching-mpls/mpls/213935-configuring-non-path-computation-element.html
- [13] Cisco. 2023. *Segment Routing Configuration Guide for Cisco NCS 5500 Series Routers, IOS XR Release 7.9.x*. Consulté le 27 janvier 2026. Disponible sur <https://www.cisco.com/c/en/us/td/docs/iosxr/ncs5500/segment-routing/79x/b-segment-routing-cg-ncs5500-79x.html>
- [14] Cisco. *Segment Routing Command Reference for Cisco 8000 Series Routers*. Consulté le 8 avril 2026. Disponible sur <https://www.cisco.com/c/en/us/td/docs/iosxr/cisco8000/segment-routing/b-segment-routing-cr-8k/segment-routing-commands.html>

Cisco DevNet Sandbox

- [15] ios-xr. 2022. *XRd-Tools*. GitHub repository. Consulté le 20 février 2026. Disponible sur <https://github.com/ios-xr/xrd-tools>
- [16] CiscoDevNet. 2024. *XRd-Sandbox*. GitHub repository. Consulté le 20 février 2026. Disponible sur <https://github.com/CiscoDevNet/XRd-Sandbox>

- [17] Cisco DevNet. *Cisco DevNet Sandbox Technical documentation - DevNet Sandbox*. Consulté le 19 février 2026. Disponible sur <https://developer.cisco.com/docs/sandbox/>
- [18] Cisco DevNet. *Cisco DevNet Sandbox Platform*. Consulté le 19 février 2026. Disponible sur <https://devnetsandbox.cisco.com/DevNet>

Cisco Modeling Labs Free (CML)

- [19] Jérôme BEZET-TORRES. 2025. *Prise en main de Cisco Modeling Labs Free (CML)*. Consulté le 10 février. Disponible sur <https://www.it-connect.fr/introduction-cisco-modeling-labs-free-cml-formez-vous-sur-de-vraies-images-cisco/>
- [20] Cisco DevNet. *Cisco Modeling Labs - Free - Cisco Modeling Labs v2.9*. Consulté le 13 février 2026. Disponible sur <https://developer.cisco.com/docs/modeling-labs/cml-free/>
- [21] Cisco DevNet. *IOL-L2 - Cisco Modeling Labs v2.9*. Consulté le 19 février 2026. Disponible sur <https://developer.cisco.com/docs/modeling-labs/iol-l2/>
- [22] Cisco. 2018. *Cisco IOS XRv 9000 Router Data Sheet*. Consulté le 19 février 2026. Disponible sur <https://www.cisco.com/c/en/us/products/collateral/routers/ios-xrv-9000-router/datasheet-c78-734034.html>

Containerlab

- [23] Romain Dodin. *containerlab - Topology definition*. Consulté le 11 février 2026. Disponible sur <https://containerlab.dev/manual/topo-def-file/>

Nokia SR Linux

- [24] Nokia. 2026. *Configuring MPLS*. Consulté le 14 février 2026. Disponible sur <https://documentation.nokia.com/srlinux/26-3/books/mpls/configuring-mpls.html>
- [25] Romain Dodin. *containerlab - Nokia SR Linux*. Consulté le 14 février 2026. Disponible sur <https://containerlab.dev/manual/kinds/srl/>
- [26] srl-labs. 2022. *Nokia SR OS SR-MPLS : low latency service with Flex-Algo*. GitHub repository, 2026. Consulté le 14 février 2026. Disponible sur <https://github.com/srl-labs/nokia-segment-routing-lab>

- [27] Nokia. *SR Linux Documentation*. Consulté le 14 février 2026. Disponible sur <https://documentation.nokia.com/srlinux/>
- [28] srl-labs. 2022. *mpls-ldp.clab.yml*. GitHub. Consulté le 16 février 2026. Disponible sur <https://github.com/srl-labs/learn-srlinux/blob/main/labs/mpls-ldp/mpls-ldp.clab.yml>

RARE/freeRtr

- [29] Romain Dodin. *containerlab - RARE/freeRtr*. Consulté le 14 mars 2026. Disponible sur <https://containerlab.dev/manual/kinds/rare-freertr/>
- [30] rare-freertr. 2018. *freeRouter source tree*. GitHub repository. Consulté le 14 mars 2026. Disponible sur <https://github.com/rare-freertr/freeRtr>
- [31] freeRtr. *freeRouter - networking swiss army knife*. Consulté le 14 mars 2026. Disponible sur <http://www.freertr.org/>
- [32] RARE Contributors. *RARE/freeRtr Documentation*. Consulté le 14 mars 2026. Disponible sur <https://docs.rare.geant.org/>

FRRouting (FRR)

- [33] Romain Dodin. *containerlab - FRR*. Consulté le 28 février 2026. Disponible sur <https://containerlab.dev/lab-examples/frr01/>
- [34] FRRouting. 2002. *FRRouting/frr*. GitHub repository. Consulté le 28 février 2026. Disponible sur <https://github.com/FRRouting/frr>
- [35] FRRouting. 2017. *Releases – FRRouting*. Consulté le 28 février 2026. Disponible sur <https://frrouting.org/release/>
- [36] FRRouting. 2017. *FRRouting User Guide — FRR latest documentation*. Consulté le 28 février 2026. Disponible sur <https://docs.frrouting.org/en/latest/>
- [37] Quay.io. *Quay – frrouting/frr*. Consulté le 28 février 2026. Disponible sur <https://quay.io/repository/frrouting/frr>
- [38] srl-labs. 2021. *containerlab/lab-examples/frr01/router1/frr.conf*. GitHub. Consulté le 28 février 2026. Disponible sur <https://github.com/srl-labs/containerlab/blob/main/lab-examples/frr01/router1/frr.conf>

VyOS

- [39] Romain Dodin. *containerlab - VyOS Networks VyOS*. Consulté le 20 février 2026. Disponible sur https://containerlab.dev/manual/kinds/vyosnetworks_vyos/
- [40] VyOS Networks. *VyOS User Guide — VyOS rolling release (current)*. Consulté le 21 février 2026. Disponible sur <https://docs.vyos.io/en/latest/>
- [41] VyOS Networks. *Segment Routing — VyOS rolling release (current)*. Consulté le 21 février 2026. Disponible sur <https://docs.vyos.io/en/latest/configuration/protocols/segment-routing.html>

Arista cEOS

- [42] Romain Dodin. *containerlab - Arista cEOS*. Consulté le 22 février 2026. Disponible sur <https://containerlab.dev/manual/kinds/ceos/>
- [43] Arista Networks. *EOS 4.36.0F User Manual*. Consulté le 23 février 2026. Disponible sur <https://www.arista.com/en/um-eos/eos-traffic-management>

OpenDayLight

- [44] OpenDaylight Projet. 2016. *Welcome to OpenDaylight Documentation — OpenDaylight Documentation Titanium documentation*. Consulté le 26 mars 2026. Disponible sur <https://docs.opendaylight.org/en/stable-titanium/>
- [45] OpenDaylight Project. 2016. *Link-State Family — BGPCEP master documentation*. Consulté le 26 mars 2026. Disponible sur <https://docs.opendaylight.org/projects/bgpcep/en/latest/bgp/bgp-user-guide-linkstate-family.html>

7. Environnement et outils

- [46] Docker. 2013. *Docker Compose*. Docker Documentation. Consulté le 11 février 2026. Disponible sur <https://docs.docker.com/compose/>
- [47] AlmaLinux OS. *About AlmaLinux Wiki | AlmaLinux Wiki*. Consulté le 2 février 2026. Disponible sur <https://wiki.almalinux.org/>
- [48] AlmaLinux OS. *AlmaLinux OS - Forever-Free Enterprise-Grade Operating System*. Consulté le 2 février 2026. Disponible sur <https://almalinux.org/fr/>
- [49] Broadcom. 2005. *VMware® Cloud Infrastructure Software*. Consulté le 9 février 2026. Disponible sur <https://techdocs.broadcom.com/us/en/vmware-cis.html>

[50] Cisco. *VMware® Cloud Infrastructure Software*. Consulté le 9 février. Disponible sur <https://techdocs.broadcom.com/us/en/vmware-cis.html>

8 Annexe ContainerLab

Avant de pouvoir utiliser ContainerLab, il est nécessaire d'installer au préalable Docker et d'être dans un environnement Linux avec les privilèges administrateur. ContainerLab s'utilise via un terminal, en utilisant la ligne de commande. Pour déployer une topologie, il faut d'abord définir un fichier de configuration au format YAML décrivant les nœuds et les liens du réseau. Après avoir configuré le fichier YAML, les commandes suivantes permettent de gérer le lab (environnement de test) :

```
containerlab deploy -t fichier.yml    # Deployer un lab
containerlab destroy -t fichier.yml   # Detruire un lab
containerlab redeploy -t fichier.yml  # Redeployer un lab
containerlab inspect                  # Inspecter l'état
containerlab graph                    # Générer un graphe
```

L'option `-t` permet de spécifier un fichier de configuration.

Tous les labs créés par Containerlab disposent d'un réseau de management qui relie les nœuds à la machine hôte via un bridge Docker nommé `docker0`, permettant ainsi de se connecter aux routeurs.

Pour capturer le trafic réseau durant nos tests, nous avons utilisé Edgeshark, un outil proposé par Containerlab, permettant de voir la topologie du réseau de conteneurs depuis l'interface web. Le service `edgeshark` se déploie via la commande :

```
curl -sL https://github.com/siemens/edgeshark/raw/main/
  deployments/wget/docker-compose.yaml | DOCKER_DEFAULT_PLATFORM=
  docker compose -f -up -d
```

Pour pouvoir lancer des captures Wireshark depuis Edgeshark, il est nécessaire d'installer un plugin Wireshark :

```
sudo dnf install ./cshargextcap_0.10.7_linux_amd64.rpm
sudo gpasswd -a $USER wireshark
```

Il suffit ensuite de cliquer sur l'aileron depuis l'interface web pour démarrer une capture wireshark. Cependant, cela ne suffit pas pour pouvoir lancer des tests. Avant de déployer un lab SR-TE, il est nécessaire d'activer MPLS sur la machine virtuelle. Il faut tout d'abord activer les modules MPLS et définir le nombre de labels disponibles :

```
# Activation des modules MPLS
sudo nano /etc/modules-load.d/mpls.conf
sudo modprobe mpls_router
sudo modprobe mpls_iptunnel
```

```
# Definition du nombre de labels
sudo nano /etc/sysctl.d/99-mpls.conf
sudo sysctl -p /etc/sysctl.d/99-mpls.conf
```

De plus, MPLS doit être activé sur les interfaces de chaque routeur. Cela doit être configuré directement dans le fichier YAML en ajoutant une section sysctls pour les labels et exec pour activer MPLS sur les interfaces :

```
topology:
  kinds:
    linux:
      image: xxxx:xxx
      sysctls:
        net.mpls.platform_labels: 100000
      exec:
        - bash -c "for i in /proc/sys/net/mpls/conf/*/input;
                    do echo 1 > $i; done"
```

Lors du démarrage d'un lab, il se peut que les routeurs FRR échouent lors de :

- L'établissement de la TED ;
- La définition de l'index pour le Segment Routing ;
- La communication BGP avec le PCE.

Ces problèmes surviennent sans qu'on ne sache pourquoi. Redéployer le lab règle souvent ce problème.

8.1 Exemple de configuration

```
mpls ip
# Definition des plages de labels
mpls label range ospf-sr 16000 8000 # SRGB : 16000 a 23999
mpls label range srlb 24000 6000   # SRLB : 24000 a 29999
mpls label range static 16 15984   # Labels statiques (reserve)
!
router ospf 1
  router-id 10.0.1.1
  # Declaration des reseaux
  network 10.0.1.1/32 area 0
  network 192.168.11.0/30 area 0
  network 192.168.12.0/30 area 0
  network 192.168.13.0/30 area 0
  network 192.168.14.0/30 area 0
```

```
# Activation du Segment Routing pour OSPF
segment-routing mpls
no shutdown
!
router traffic-engineering
# Configuration du Traffic Engineering
segment-routing
explicit-null ipv4
```

Configuration d'un routeur Arista cEOS

```
router ospf
  ospf router-id 10.0.0.1
  ospf opaque-lsa
  !
  # Activation du Segment Routing pour OSPF
  segment-routing on
  # Definition de la plage de labels
  segment-routing global-block 16000 23999 local-block
    15000 15999
  segment-routing prefix 10.0.0.1/32 index 1 no-php-flag
  !
  network 192.168.12.0/30 area 0
  network 192.168.14.0/30 area 0
  network 192.168.13.0/30 area 0
  network 192.168.10.0/30 area 0
  network 10.0.0.1/32 area 0
  !
  #Activation de MPLS-TE
  mpls-te on
  mpls-te router-address 10.0.0.1
  mpls-te export
  exit
!
```

Configuration d'OSPF dans un routeur FRR

```
segment-routing
  traffic-eng
  mpls-te on
  mpls-te import ospfv2
  segment-list label_list
    index 2 mpls label 16002
    index 3 mpls label 16003
    index 4 mpls label 16004
```

Création d'une liste de segment (chemin) dans FRR

```
segment-routing
  traffic-engineering
    # Definition de la SR Policy (identifiant + destination)
    policy color 100 endpoint 10.0.0.4
    # Definition du chemin candidat pour la policy
    candidate-path preference 20 name CP1 explicit segment-
      list label_list
```

Création d'une SR-Policy utilisant le chemin créé

```
segment-list nai_list2
  index 2 nai adjacency 192.168.12.1 192.168.12.2
  index 4 nai prefix 10.0.0.4/32 algorithm 0
```

Utilisation des Adjacency IDs pour définir un chemin

9 Annexe IOS XR

Cette annexe regroupe les commandes et sorties principales utilisées pour valider la partie IOS-XRd. Les détails complets ne sont pas tous reproduits afin de garder l'annexe lisible. L'objectif est de conserver les preuves nécessaires pour vérifier le lancement de la topologie, les SR Policies explicites, les SR Policies dynamiques, le fonctionnement du PCE stateful et la limite rencontrée sur la contrainte de bande passante.

9.1 Accès au sandbox XRd

L'accès au sandbox XRd se fait depuis la plateforme Cisco DevNet Sandbox [18]. Il nécessite une réservation du lab, une connexion au VPN Cisco DevNet, puis une connexion SSH à la machine virtuelle fournie par Cisco.

L'adresse VPN, l'adresse de la machine virtuelle et les identifiants SSH sont fournis directement sur la page de présentation du lab après sa réservation sur Cisco DevNet Sandbox.

```
# Connexion au VPN Cisco DevNet
sudo openconnect <adresse_vpn_fournie_par_devnet>

# Suppression d'une ancienne cle SSH si necessaire
ssh-keygen -R <adresse_vm_fournie_par_devnet>

# Connexion a la VM du sandbox
```

```
ssh <utilisateur_fourni_par_devnet>@<
  adresse_vm_fournie_par_devnet >
```

Une fois connecté à la machine virtuelle, les routeurs et les hôtes sont lancés sous forme de conteneurs Docker. L'accès aux routeurs IOS-XRd se fait principalement avec `docker attach` ou `docker exec`.

```
# Afficher les conteneurs actifs
docker ps

# Accéder à un routeur XRd
docker attach router1

# Accéder à un hôte de test
docker exec -it pc1 sh
docker exec -it pc2 sh
```

9.2 Lancement de la topologie adaptée

La topologie fournie par défaut dans le sandbox Cisco a d'abord été étudiée, mais elle n'a pas été utilisée telle quelle car elle reposait sur une organisation différente de celle attendue dans le projet. Une topologie adaptée a donc été construite à partir des fichiers de configuration du projet.

Les fichiers nécessaires au lancement de la topologie sont stockés dans la machine virtuelle AlmaLinux, dans le répertoire `~/Documents/Cisco/config`. Ce répertoire contient notamment le script de déploiement, les fichiers Docker Compose et les fichiers de configuration de démarrage des routeurs.

Depuis la machine virtuelle AlmaLinux, les fichiers sont copiés vers la machine virtuelle distante du sandbox avec la commande suivante :

```
scp -r ~/Documents/Cisco/config \
  <utilisateur_fourni_par_devnet>@<adresse_vm_fournie_par_devnet>:
  >:/home/<utilisateur_fourni_par_devnet>/
```

L'adresse de la machine virtuelle distante et les identifiants SSH sont indiqués sur la page de présentation du lab après sa réservation sur Cisco DevNet Sandbox.

Depuis la machine virtuelle distante du sandbox, il faut ensuite se placer dans le répertoire copié, puis lancer le script de déploiement :

```
cd ~/config
bash deploy.sh
```

Le script `deploy.sh` permet de nettoyer les anciens conteneurs, de générer ou d'utiliser le fichier `docker-compose.yml`, puis de démarrer la topologie IOS-XRd adaptée au projet.

9.3 Commandes de vérification rapide

Après le démarrage de la topologie, quelques commandes permettent de vérifier rapidement que les éléments principaux sont actifs.

Sur router1 :

```
show ospf neighbor
show segment-routing traffic-eng pcc ipv4 peer
show bgp vrf 100 ipv4 unicast 10.3.1.0/24
```

Sur le PCE :

```
show pce peer
show pce lsp
show pce lsp detail
```

Depuis pc1, la connectivité vers pc2 peut être testée avec :

```
ping 10.3.1.2
traceroute 10.3.1.2
```

9.4 Validation des SR Politiques explicites

L'objectif de ce test est de vérifier que router1 peut imposer un chemin précis sans demander de calcul au PCE. Deux politiques explicites ont été testées :

- **color 200** : chemin haut via router2;
- **color 300** : chemin bas via router3.

Les politiques configurées sur router1 sont :

```
srte_c_200_ep_100.100.100.104
srte_c_300_ep_100.100.100.104
```

Les commandes utilisées pour vérifier les politiques sont :

```
show segment-routing traffic-eng policy name srte_c_200_ep_100
.100.100.104
show segment-routing traffic-eng policy name srte_c_300_ep_100
.100.100.104

show segment-routing traffic-eng forwarding policy \
color 200 endpoint ipv4 100.100.100.104

show segment-routing traffic-eng forwarding policy \
color 300 endpoint ipv4 100.100.100.104
```

Pour la couleur 200, la table de forwarding montre que le trafic sort vers router2 :

```
Color: 200, End-point: 100.100.100.104
Name: srte_c_200_ep_100.100.100.104
Candidate path: SRTE-EXPLICIT-TOP
SL[0]: EXPLICIT-VIA-R2
Outgoing Interfaces: GigabitEthernet0/0/0/0
Next Hop: 100.101.102.102 --> router2
Label Stack (Top -> Bottom): { 16104 }
```

Pour la couleur 300, la table de forwarding montre que le trafic sort vers router3 :

```
Color: 300, End-point: 100.100.100.104
Name: srte_c_300_ep_100.100.100.104
Candidate path: SRTE-EXPLICIT-BOT
SL[0]: EXPLICIT-VIA-R3
Outgoing Interfaces: GigabitEthernet0/0/0/1
Next Hop: 100.101.103.103 --> router3
Label Stack (Top -> Bottom): { 16104 }
```

Le label 16104 correspond au Prefix-SID de router4. La différence entre les deux politiques se voit donc surtout dans l'interface de sortie et le next-hop.

Pour déclencher temporairement la policy explicite, la couleur exportée par router4 a été modifiée. Exemple pour la couleur 200 :

```
conf t
extcommunity-set opaque COLOR_200
  200
end-set

route-policy SET_COLOR_200
  set extcommunity color COLOR_200
  pass
end-policy

vrf 100
  address-family ipv4 unicast
    export route-policy SET_COLOR_200
  !
!
commit
end
```

La route reçue sur router1 confirme que la couleur 200 est bien associée à la route vers pc2 :

```
show bgp vrf 100 ipv4 unicast 10.3.1.0/24
```

```
Extended community: Color:200 RT:100:100  
SR policy color 200, up, not-registered, bsid 24006
```

Les traceroute depuis pc1 confirment les chemins suivis :

```
# Color 200 : chemin haut  
1  router1.config_pc1-router1 (10.1.1.3)  
2  100.101.102.102 --> router2  
3  100.102.104.104 --> router4  
4  10.3.1.2 --> pc2  
  
# Color 300 : chemin bas  
1  router1.config_pc1-router1 (10.1.1.3)  
2  100.101.103.103 --> router3  
3  100.103.104.104 --> router4  
4  10.3.1.2 --> pc2
```

Les politiques explicites sont donc validées : le trafic suit bien le chemin imposé par la configuration.

9.5 Validation d'une policy dynamique avec métrique IGP

Ce test vérifie qu'une route annoncée avec la couleur 100 déclenche automatiquement une SR Policy dynamique sur router1. Le chemin est ensuite calculé par le PCE avec la métrique IGP.

La route BGP reçue sur router1 contient bien la couleur 100 :

```
show bgp vrf 100 ipv4 unicast 10.3.1.0/24  
  
Extended community: Color:100 RT:100:100  
SR policy color 100, up, registered, bsid 24008
```

La policy dynamique apparaît comme active et calculée par le PCE :

```
show segment-routing traffic-eng policy color 100 \  
    endpoint ipv4 100.100.100.104  
  
Color: 100, End-point: 100.100.100.104  
Status: Admin: up Operational: up  
Preference: 100 (BGP ODN) (active)  
Dynamic (pce 100.100.100.107) (valid)  
Metric Type: IGP, Path Accumulated Metric: 21  
SID[0]: 16104 [Prefix-SID, 100.100.100.104]  
Binding SID: 24008
```

La présence du chemin côté PCE est vérifiée avec :


```
show pce lsp

PCC 100.100.100.101:
Tunnel Name: bgp_c_100_ep_100.100.100.104_discr_100
State: Admin up, Operation active
Setup type: Segment Routing
Binding SID: 24008
```

Le trafic depuis pc1 suit le chemin via router2 :

```
traceroute 10.3.1.2

1  router1.config_pc1-router1 (10.1.1.3)
2  100.101.102.102 --> router2
3  100.102.104.104 --> router4
4  10.3.1.2 --> pc2
```

La policy dynamique color 100 est donc validée : elle est créée automatiquement, calculée par le PCE, et le trafic suit le chemin attendu.

9.6 Validation d'une policy dynamique avec métrique de latence

Ce test vérifie qu'une SR Policy dynamique peut être calculée avec un critère différent de la métrique IGP. La couleur 128 est utilisée pour déclencher une policy calculée avec une métrique de latence.

La couleur 128 est d'abord exportée par router4 :

```
conf t
extcommunity-set opaque COLOR_128
  128
end-set

route-policy SET_COLOR_128
  set extcommunity color COLOR_128
  pass
end-policy

vrf 100
  address-family ipv4 unicast
    export route-policy SET_COLOR_128
  !
!
commit
end
```

La route BGP reçue sur router1 montre bien la couleur 128 :

```
show bgp vrf 100 ipv4 unicast 10.3.1.0/24

Extended community: Color:128 RT:100:100
SR policy color 128, up, registered, bsid 24010
```

La policy SR-TE associée est active et calculée par le PCE avec la métrique LATENCY :

```
show segment-routing traffic-eng policy color 128 detail

Color: 128, End-point: 100.100.100.104
Status: Admin: up Operational: up
Preference: 100 (BGP ODN) (active)
Dynamic (pce 100.100.100.107) (valid)
Metric Type: LATENCY, Path Accumulated Metric: 20
SID[0]: 16104 [Prefix-SID, 100.100.100.104]
Binding SID: 24010
```

Côté PCE, le calcul est également visible :

```
show pce lsp detail

Tunnel Name: bgp_c_128_ep_100.100.100.104_discr_100
Color: 128
State: Admin up, Operation active
Setup type: Segment Routing
Binding SID: 24010

Reported path:
  Metric type: LATENCY (12), Accumulated Metric 20

Computed path: (Local PCE)
  Metric type: LATENCY (12), Accumulated Metric 20
```

Dans le test observé, le trafic passe par router3 :

```
traceroute 10.3.1.2

1  router1.config_pc1-router1 (10.1.1.3)
2  100.101.103.103 --> router3
3  100.103.104.104 --> router4
4  10.3.1.2 --> pc2
```

La policy color 128 est donc validée : elle est active, calculée par le PCE avec une métrique de latence, et permet d'orienter le trafic vers un chemin différent.

9.7 Vérification du PCE stateful

Ce test vérifie que le PCE fonctionne en mode stateful. Dans ce mode, il ne calcule pas seulement un chemin : il garde aussi l'état des chemins actifs et peut échanger des mises à jour avec les routeurs.

Sur router1, la session PCEP avec le PCE est active :

```
show segment-routing traffic-eng pcc ipv4 peer detail

Peer address: 100.100.100.107
State up
Capabilities: Stateful, Update, Segment-Routing, Instantiation
Report messages: tx 5
Update messages: rx 1
```

Côté PCE, les routeurs connectés apparaissent également en état Up :

```
show pce ipv4 peer detail

Peer address: 100.100.100.101
State: Up
Capabilities: Stateful, Segment-Routing, Update, Instantiation
```

La base LSP du PCE permet de vérifier que les chemins SR-TE actifs sont connus par le contrôleur :

```
show pce lsp
show pce lsp detail
```

Une capture Wireshark a aussi été réalisée pendant une coupure temporaire du lien entre router1 et router2. La capture montre des échanges PCRpT puis PCUpd, ce qui confirme que le PCE reçoit les changements et peut participer à la mise à jour des politiques dynamiques.

9.8 Étude de la contrainte de bande passante

Ce test avait pour objectif de vérifier si le PCE pouvait calculer un chemin en tenant compte d'une contrainte de bande passante minimale.

Les informations de bande passante étaient bien visibles côté PCE :

```
Bandwidth: Total 125000000 Bps, Reservable 12500000 Bps
```

Cela montre que les informations de capacité des liens étaient bien remontées. La difficulté se situait donc au moment d'appliquer réellement une contrainte de bande passante à une SR Policy dynamique.

Une policy dynamique de test a été utilisée :

```
segment-routing
traffic-eng
policy TEST-BW
  color 500 end-point ipv4 100.100.100.104
  candidate-paths
    preference 100
    dynamic
      pcep
      !
      metric
        type igp
      !
    !
  !
!
```

La policy est bien calculée par le PCE avec une métrique IGP :

```
Dynamic (pce 100.100.100.107) (valid)
Metric Type: IGP
Path Accumulated Metric: 21
SID[0]: 16104
```

La contrainte de bande passante a ensuite été testée avec `override-rules` côté PCE, selon la syntaxe documentée par Cisco [14] :

```
pce
  override-rules
    sequence 1
      matching-criteria
        peer all
        lsp
        colors 500
      !
    !
  override
    constraints
      bandwidth 1000000
    !
  !
!
```

Cependant, les traces PCEP montrent que la bande passante transmise reste à 0 kbps :

```
Sending PCUpd SRTE, [PLSP-ID:3]
req-bw/res-bw:0/0 kbps

Received PCRpt.
req-bw/res-bw:0/0 kbps flags:(C:0)
```

La règle d'override correspond bien à la policy visée, mais la valeur de bande passante n'est pas réellement appliquée. Aucun recalcul basé sur cette contrainte n'a donc pu être validé avec la version IOS-XRd 25.3.1 utilisée.

Pour vérifier que le PCE pouvait tout de même recalculer un chemin avec une contrainte prise en compte, un recalcul a été testé en modifiant le coût OSPF du chemin haut.

```
# Sur router1, lien vers router2
router ospf 1
 area 0
   interface GigabitEthernet0/0/0/0
     cost 1000

# Sur router2, lien vers router4
router ospf 1
 area 0
   interface GigabitEthernet0/0/0/1
     cost 1000
```

Après modification, le trafic passe par router3 :

```
traceroute 10.3.1.2

1  router1.config_pc1-router1 (10.1.1.3)
2  100.101.103.103 --> router3
3  100.103.104.104 --> router4
4  10.3.1.2         --> pc2
```

Cette vérification montre que le recalcul dynamique fonctionne lorsque la contrainte utilisée est correctement prise en compte. La limite observée concerne donc surtout l'application de la contrainte de bande passante avec `override-rules bandwidth` dans l'environnement IOS-XRd testé.

9.9 Synthèse des validations IOS-XRd

Validation	Résultat observé	Statut
Policies explicites	Chemins imposés correctement via router2 ou router3 selon la couleur utilisée	Validé
Policy dynamique IGP	Policy créée automatiquement par BGP ODN et calculée par le PCE avec la métrique IGP	Validé
Policy dynamique latence	Policy calculée par le PCE avec la métrique LATENCY	Validé
PCE stateful	Sessions PCEP actives, base LSP visible et échanges PCRpt/PCUpd observés	Validé
Bande passante dans la topologie	Informations de capacité visibles côté PCE	Validé
Contrainte de bande passante	La contrainte reste envoyée à 0 kbps avec override-rules bandwidth	Partiel
Recalcul par métrique IGP	Le trafic bascule vers router3 après modification du coût OSPF	Validé